CALCULATOARE NUMERICE

Calculatoare 2 2024-2025 Sem. 2

CURS 10 Sapt 10 8 mai 2025 12:00-14:00

serban@upit.ro

Regulament disciplina

Nota finala este formata din activitatile:

- Laborator 30%
- Lucrare control (midterm) 20%
- Examen 50%
- Bonus – prezenta activa curs 10%

Conditii pentru promovare:

- Nota de la laborator trebuie sa fie minim 5 (prezenta obligatorie la toate sedintele de laborator);
- Nota la lucrarea de control trebuie sa fie minim 5;
- Nota de la examen trebuie sa fie minim 5.

In cazul reluarii disciplinei intr-un alt an universitar, activitatile nepromovate trebuie parcurse din nou.

Examen 19 iunie 2025 **ambele grupe simultan**

Lucrare de control **15 mai 2025 ora 12**

Continut disciplina

Structuri digitale folosite în calculatoare numerice

Memorii semiconductoare in calculatoare numerice

Memoria cache

Memoria virtuală

Adresarea memoriei in sistemele de calcul

Bibliografie

1. **David PATTERSON, John HENNESSY** *Computer Organization and Design The Hardware/Software Interface,* 5th ed., Morgan Kaufmann Elsevier 2012 (a se vedea şi traducerea în lb. română David A. Patterson, John L. Hennessy *Organizarea şi proiectarea calculatoarelor, Interfaţa hardware/software*; Editura All, Bucureşti, 2002);
2. John L. HENNESSY, David A. PATTERSON *Computer Architecture, A Quantitative Approach*, 5th ed., Morgan Kaufmann Publishers, Inc, San Francisco, 2012;
3. Bruce JACOB, Spencer NG, David WANG *Memory Systems Cache, DRAM, Disk* Morgan Kaufmann Elsevier 2008;
4. Miles MURDOCCA, Vincent HEURING *Principles of Computer Architecture*, Pretice Hill, 1999;
5. Jim HANDY The Cache Memory, 2nd ed., Academic Press Elsevier, 1998;
6. **Andrew S. TANENBAUM, Todd AUSTIN** *Structured Computer Organization*, 6th ed., Prentice-Hall, Inc., 2013 (a se vedea şi traducerea în lb. română Andrew S. Tanenbaum *Organizarea Structurată a Calculatoarelor*, Agora, Tg. Mureş, 2004);
7. **William STALLINGS** *Computer Organization and Architecture: Designing for Performance*, 9th edition, Prentice-Hall Inc., 2013;
8. Vincent HEURING, Harry JORDAN *Computer Systems Design and Architecture*, 2nd ed., Person Prentince Hill, 2007;
9. Sajjan G. SHIVA *Computer Organization, Design, and Architecture,* 4th ed., CRC Press, Taylor & Francis Group, Boca Raton, USA, 2008;
10. Carl HAMACHER, Zvonko VRANESIC, Safwat ZAKY, Naraig MANJIKIAN *Computer organization and embedded systems, 6th edition* McGraw Hill, 2012

Circuite de memorie semiconductoare

Exista doua tipuri mari de memorii: ROM (read only memory) si RAM (random access memory/ read-write memory).

Exista doua mari tehnologii prin care circuitele de memorie se pot realiza:

→Tehnologia bipolara – foloseste tranzistoare npn, pnp

→Tehnologia unipolara – foloseste tranzistoare nMOS, pMOS (CMOS)

Circuitele de memorie realizate in tehnologie bipolara se caracterizeaza prin urmatoarele:

→ Timp de raspuns / acces foarte scazut (sunt foarte rapide);

→ Densitate mica in integrare;

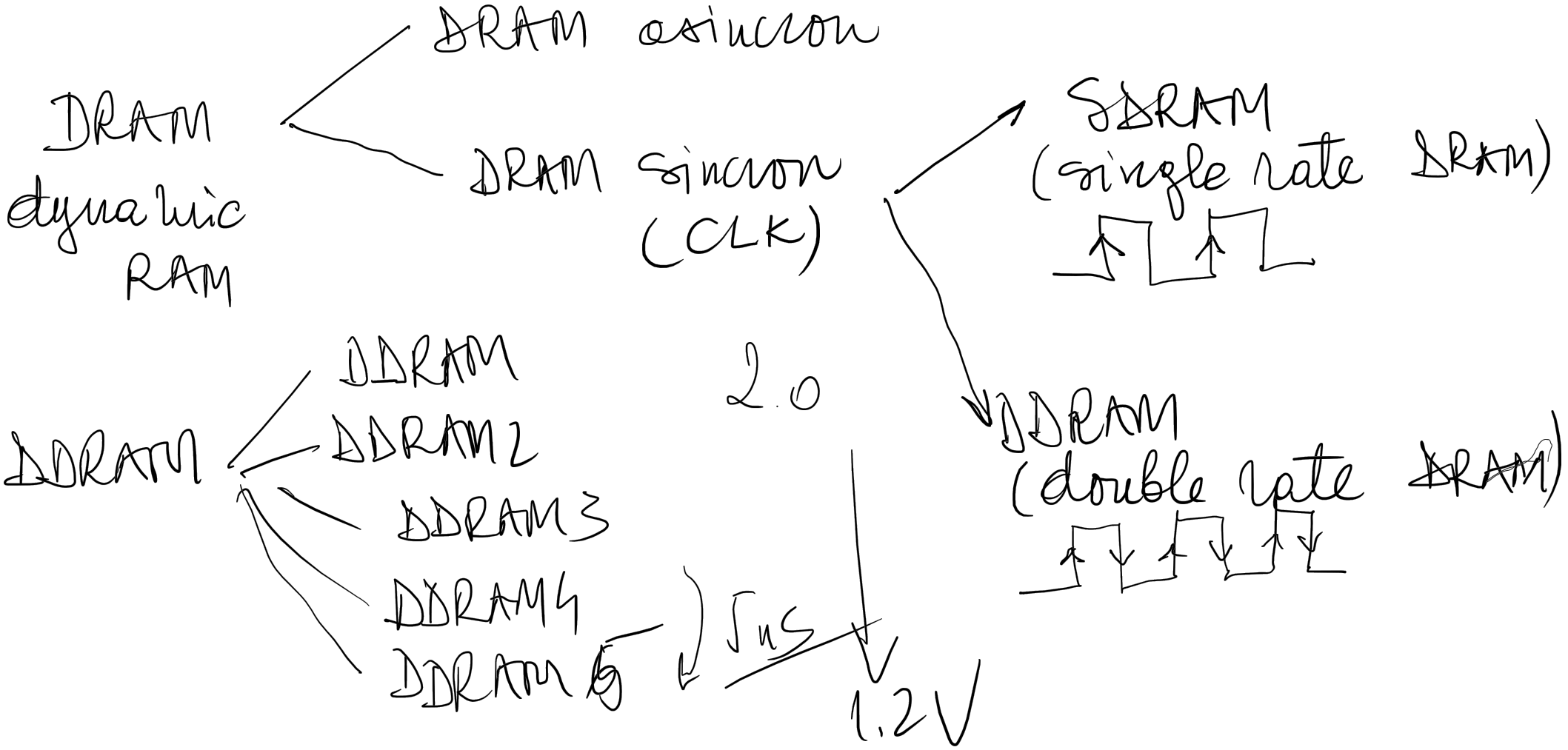
→ Consum energetic ridicat.

Circuitele de memorie unipolare prezinta urmatoarele caracteristici:

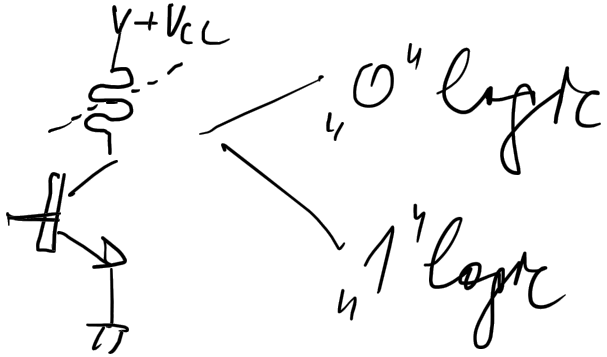
→ Densitate de integrare foarte mare, prezinta capacitate foarte mari;

→ Consum energetic redus;

→ Timpul de acces este crescut (nu sunt foarte rapide).

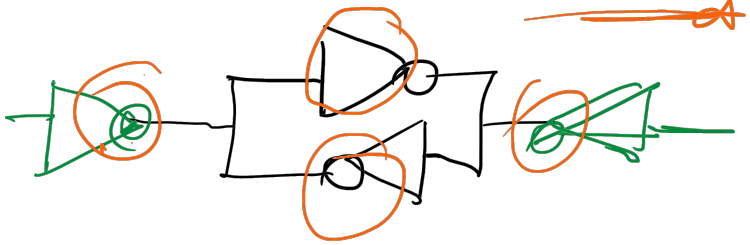

DRAM
dynamic
RAM
DRAM asincron
DRAM sincron
(CLK)
SDRAM
(single rate DRAM)
DDRAM
(double rate DRAM)
DDRAM
DDRAM
DDRAM2
DDRAM3
DDRAM4
DDRAM5
2.0
1.2V

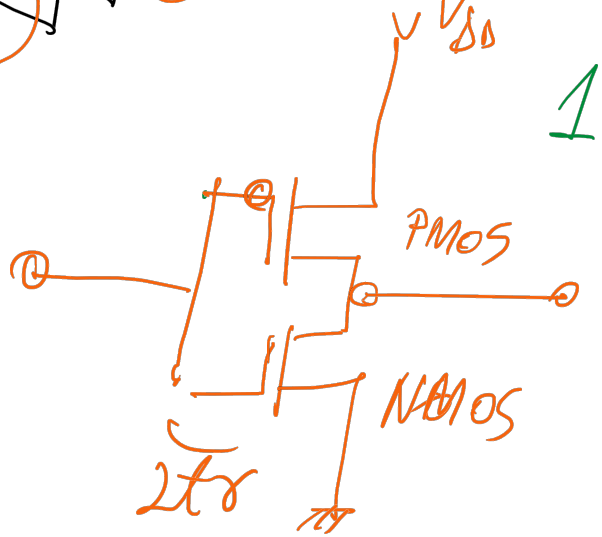
Celula' memorie in ROM

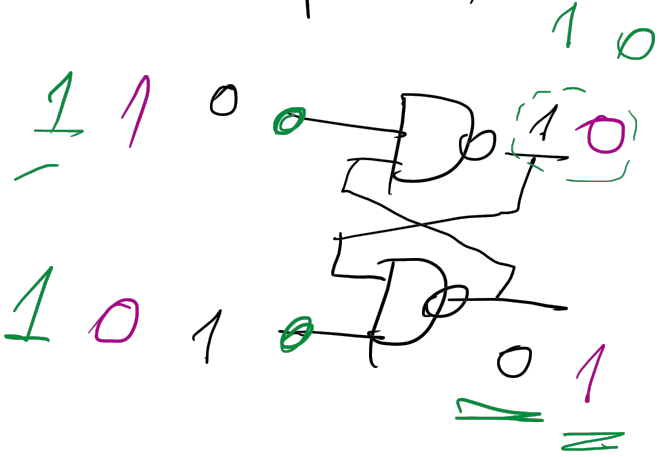


RAM
- SRAM
- DRAM (unipolare)

SRAM (unipolar)







TIPURI DE MEMORII ROM UNIPOLARE

1. PROM (programmable ROM / OTP (one time programming)) - nu admit stergeri;
2. EPROM (erasable PROM) – admit stergere si se pot reprograma. Cipul prezinta o fereasta de sticla transparenta, stergerea se face prin expunere la raze ultraviolete. Numarul de cicli de stergere se apropie de 50;
3. EEPROM (electrically EPROM/ flash ROM) - memorii care se sterg electric. Numarul de stergeri urca peste 10000. Admit programare inCircuit.
4. Serial EEPROM adresa se da bit dupa bit. Numarul de stergeri urca la 1000000.

TIPURI DE MEMORII RAM

Memorii RAM statice – SRAM (Static RAM) – bipolare, unipolare

Memorii RAM dinamice – DRAM – unipolara

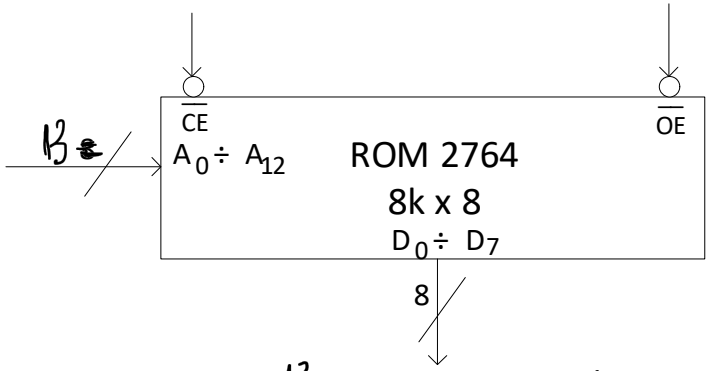
DRAM – 2 tipuri – asincrone, sincrone

DRAM sincrone: **SDRAM (single data rate DRAM - operatii pe un singur front al CLK)**
DDRAM (double data rate DRAM – operatii pe ambele fronturi ale CLK)

DDRAM: DDRAM 2, 3, 4, 5, 6

END 27 mar 2025

$\overline{CE}$ - chip enable $\overline{OE}$ - output enable



Memorii ROM

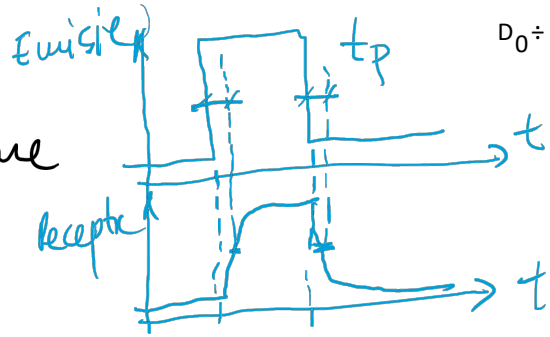
$$R = \rho \cdot \frac{l}{S}$$

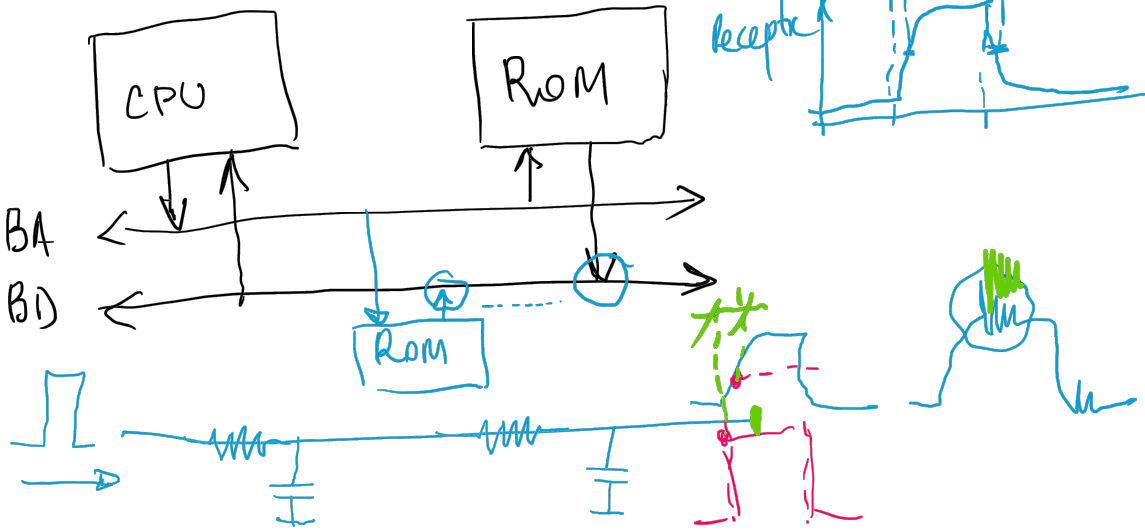
$8K = 2^3 \cdot 2^{10} = 2^{13} \Rightarrow$ 13 linii adrese $A_0 \div A_{12}$

$8 \Rightarrow$ 8 linii de date $D_0 \div D_7$

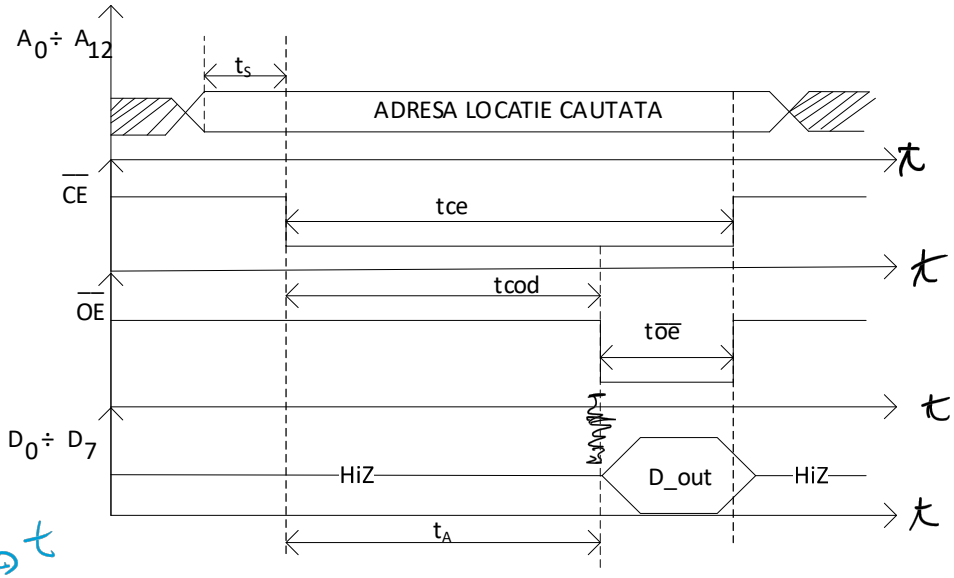
$8K \times 8 = 64Kb \rightarrow$ capacitate

$8K \times 8 = 8K_8 = 8KB$ - organizare





operație citire locație ROM



t_s - set-up $\geq$ 5ns

t_{CE} (chip enable) $\geq$ 150 ns

t_{COD} ($\overline{CE}$ - $\overline{OE}$ - delay) $\geq$ 50 ns

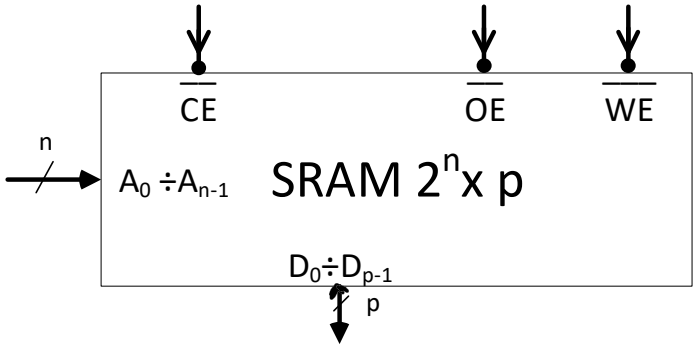
t_A (acces) $\geq$ 120 ns

$f_{CPU} = 1\,GHz \Rightarrow T_{CPU} = 1\,ns$

instructiune executată - 10 ÷ 20 ns

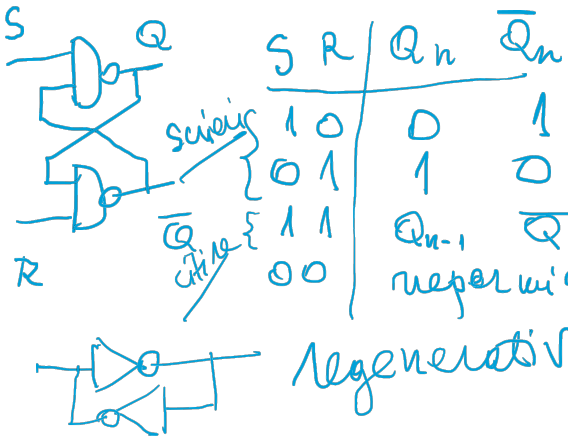
Memorii SRAM

WE - write enable



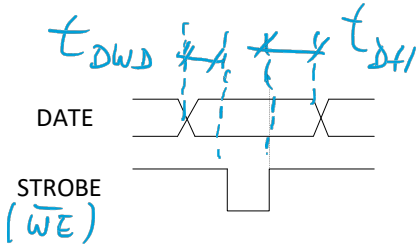
Read

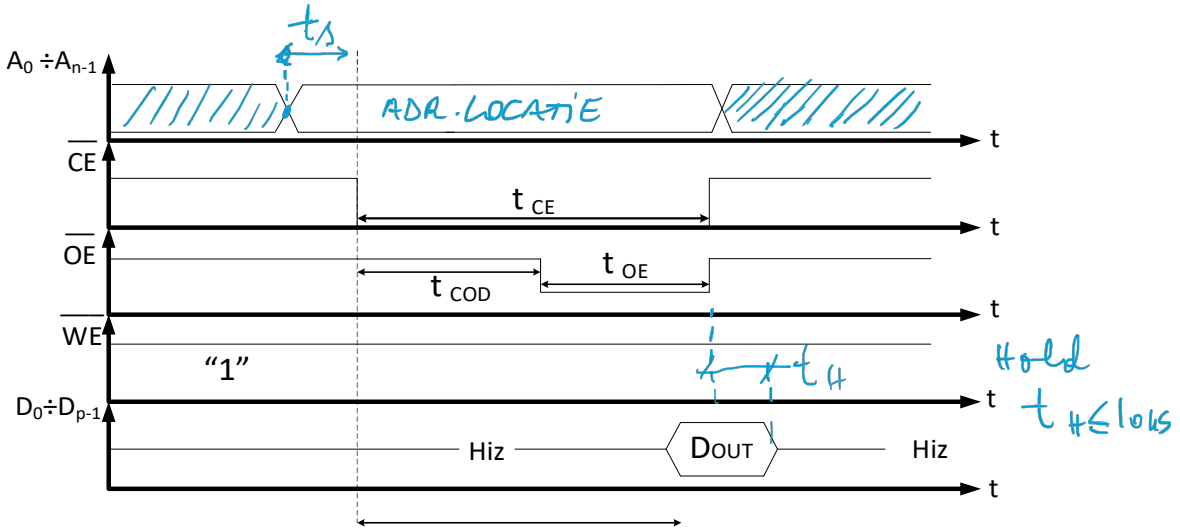
SRAM 6264 8k x 8

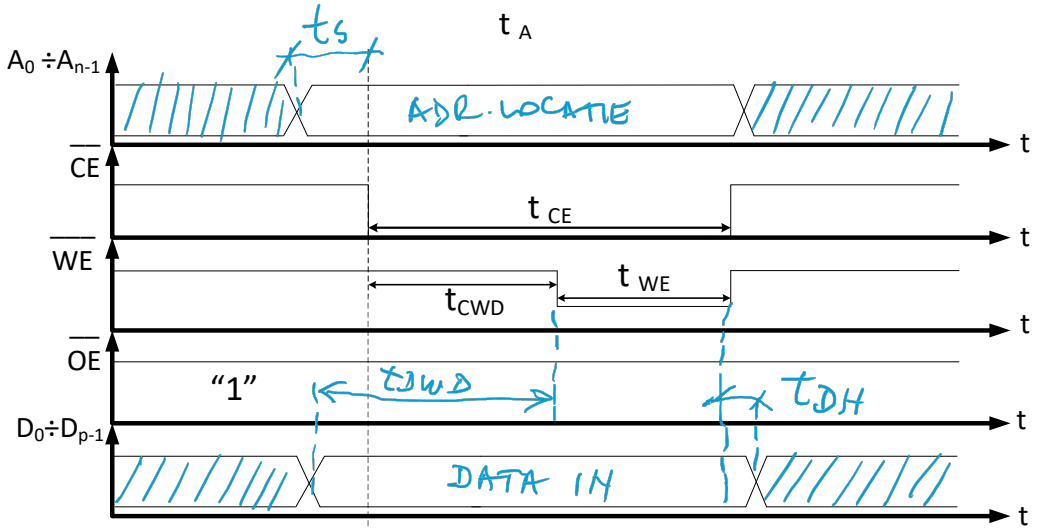


regenerativ

Write







t_{CWS} - CE - WE - delay $\geq$ 50ns $t_{WE} \geq$ 25ns

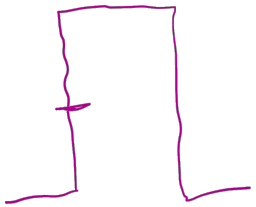
t_{DWS} - DATA WRITE DELAY $\geq$ 15ns

t_{DH} - DATA HOLD $\geq$ 15ns

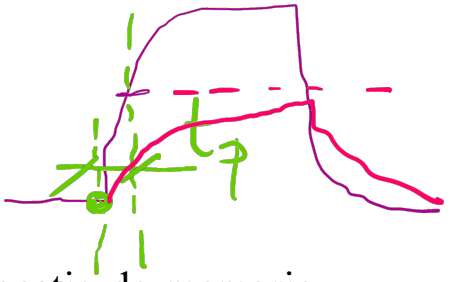
Conectarea circuitelor de memorie la magistralele sistemelor de calcul

Principii folosite la conectarea circuitelor de memorie la magistralele sistemelor de calcul:

- Conectarea trebuie facuta astfel incat la o adresa emisa de procesor sa raspunda cel mult o locatie de memorie. Daca ar raspunde 2 locatii de memorie diferite sistemul nu ar functiona corect.
- O locatie de memorie poate sa raspunda la cel putin o adresa emisa de procesor. Cand raspunde la exact o adresa a procesorului, tehnica de adresare este una de tip complet.
- Cand raspunde la 2 sau mai multe adrese de memorie, tehnica de adresare este incompleta
- Memoriile trebuiesc conectate astfel incat ele sa corespunda din punct de vedere al datelor cu numarul de biti aflati in magistrala de date a sistemului de calcul in care sunt conectate
- Conectarea trebuie sa fie facuta astfel incat magistralele de adrese si de date sa nu fie incarcate nici rezistiv, nici capacitiv.
- Cand mai multe memorii sunt conectate la magistrala de date, logica de decodare si selectie trebuie sa actioneze astfel incat doar etajele de iesire ale unui singur circuit de memorie sa ramana activ, celelalte ramanand in HiZ.
- Conectarea memoriile ROM trebuie facuta astfel incat eventualele operatii accidentale de scriere in respectivul circuit sa fie blocate. La memoria RAM nu trebuie sa apara o astfel de conditionare.

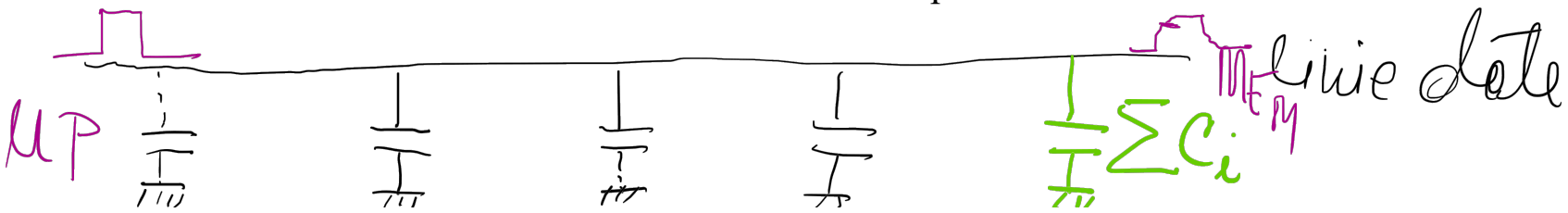


Conectarea circuitelor de memorie la magistralele sistemelor de calcul



Principii folosite la conectarea circuitelor de memorie la magistralele sistemelor de calcul:

- Conectarea trebuie facuta astfel incat la o adresa emisa de procesor sa raspunda cel mult o locatie de memorie. Daca ar raspunde 2 locatii de memorie sistemul nu ar functiona corect.
- O locatie de memorie poate sa raspunda la cel putin o adresa emisa de procesor. Cand raspunde la exact o adresa a procesorului, tehnica de adresare este una de tip complet.
- Cand raspunde la 2 sau mai multe adrese de memorie, tehnica de adresare este incompleta
- Memoriile trebuiesc conectate astfel incat ele sa corespunda din punct de vedere al datelor cu numarul de biti aflati in magistrala de date a sistemului de calcul in care sunt conectate
- Conectarea trebuie sa fie facuta astfel incat magistralele de adrese si de date sa nu fie incarcate nici rezistiv, nici capacitiv.
- Cand mai multe memorii sunt conectate la magistrala de date, logica de decodare si selectie trebuie sa actioneze astfel incat doar etajele de iesire ale unui singur circuit de memorie sa ramana activ, celelalte ramanand in HiZ.
- Conectarea memoriile ROM trebuie facuta astfel incat eventualele operatii accidentale de scriere in respectivul circuit sa fie blocate. La memoria RAM nu trebuie sa apara o astfel de conditionare.



END 3 apr 2025

Problema. Intr-un sistem de calcul magistrala de adrese este de 16 biti, iar magistrala de date de 8 biti. In acest sistem de calcul trebuie conectata o zona de memorie ROM de 24KB, folosind circuite de memorie ROM de tipul 2764 cu organizarea 8K x 8. Adresa de inceput a zonei de memorie ROM este sa fie 2000h, zona de memorie ROM este continua, iar adresarea este completa. Prezentati proiectarea conectarii circuitelor de memorie pentru indeplinirea cerintelor.

Metodologie de rezolvare:

1. Prezentarea circuitului cu care se realizeaza implementarea.
2. Determinarea numarului de circuite pentru realizarea zonei de memorie.
3. Intocmirea tabelului de adresare a zonei de memorie.
4. Prezentarea schemei bloc a structurii ce trebuie realizata.
5. Proiectarea circuitului de decodificare si selectie
6. Conectarea circuitelor de memorie ROM.

Solutie

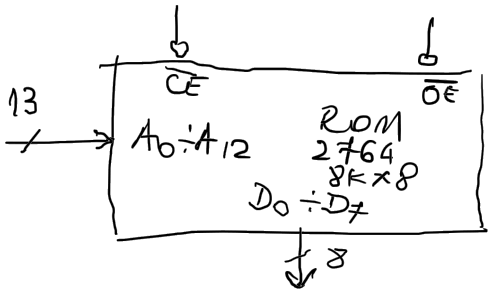
1. ROM 2764 $8K \times 8 = 8K_0 = 8KB$ (organizare)

Capacitate: $8K \times 8 = 64Kb$

* Nr. linii adrese: $8 \cdot K = 2^3 \cdot 2^{10} = 2^{13}$

13 adrese $A_0 \div A_{12}$

* Nr. linii date: 8 $D_0 \div D_7$



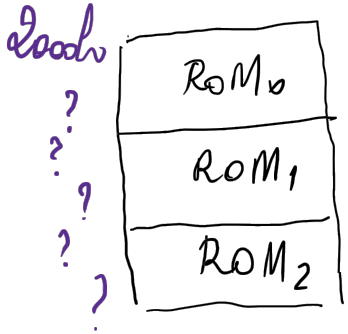
2. organizare zona $\frac{24K \times 8}{8K \times 8} = 3 \times 1 = 3$ CI utilizate

3 (linii), 1 (coloane)

organizare circuit 1 1

matrice circuite utilizate

Dodo



ROM0
ROM1
ROM2

3. Magistrala de adrese 16 biți : $BA_0 \div BA_{15}$ (Bus) ROM 2764 - 8KB $8K = 2^{13}$

$2^{13} - 1 \Rightarrow$ ← nr locații din circuitul ROM

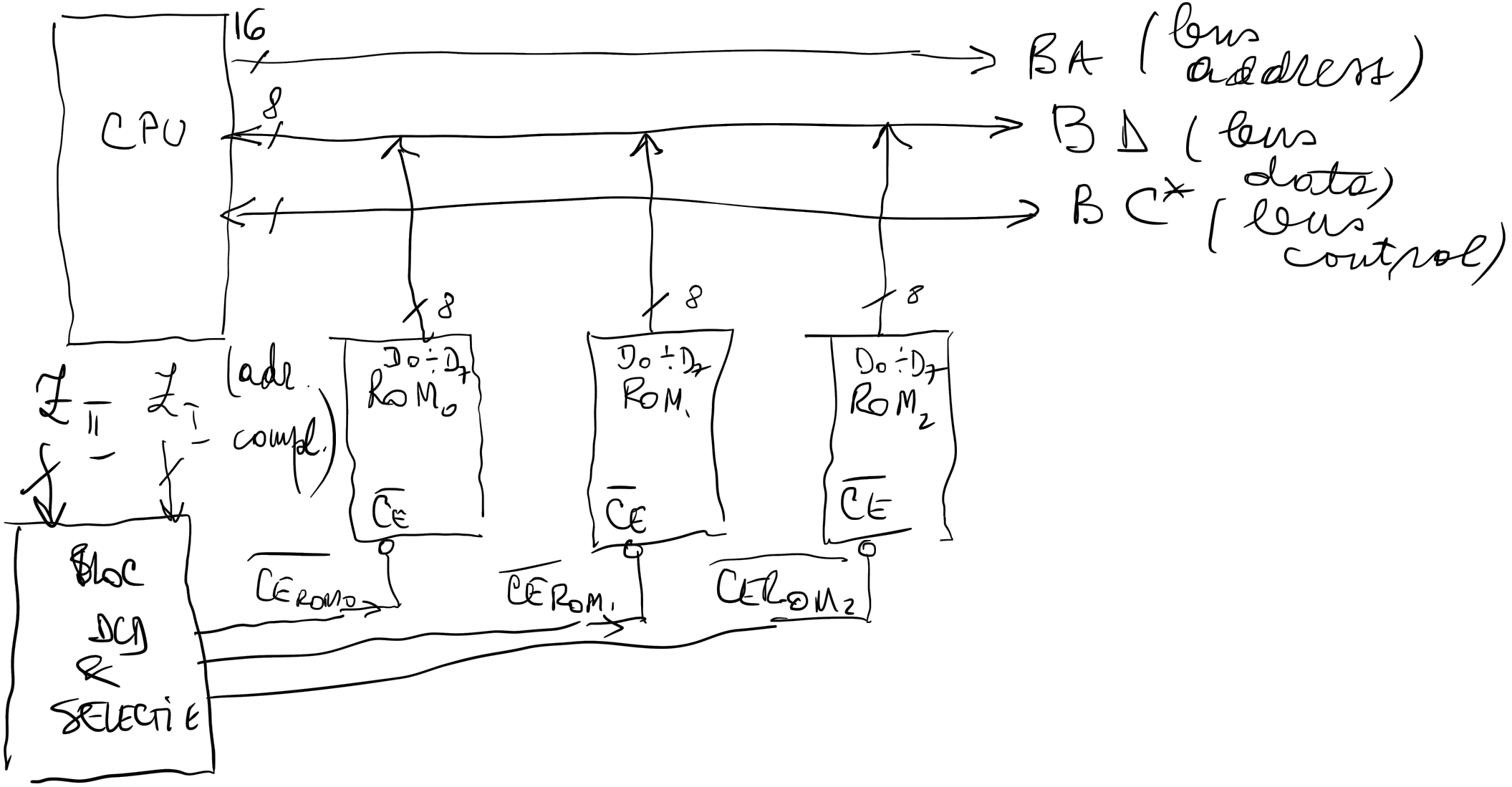
BA_{15}	BA_{14}	BA_{13}	BA_{12}	BA_{11}	BA_{10}	BA_9	BA_8	BA_7	BA_6	BA_5	BA_4	BA_3	BA_2	BA_1	BA_0	ADRESA HEXA	Circuit
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000h	ROM_0
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFFh	
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000h	ROM_1
0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	5FFFh	
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	6000h	ROM_2
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	7FFFh	

A000h – BFFFh } $BA_{15} = 1$.
C000h – DFFFh } —//—
E000h – FFFFh } —//—

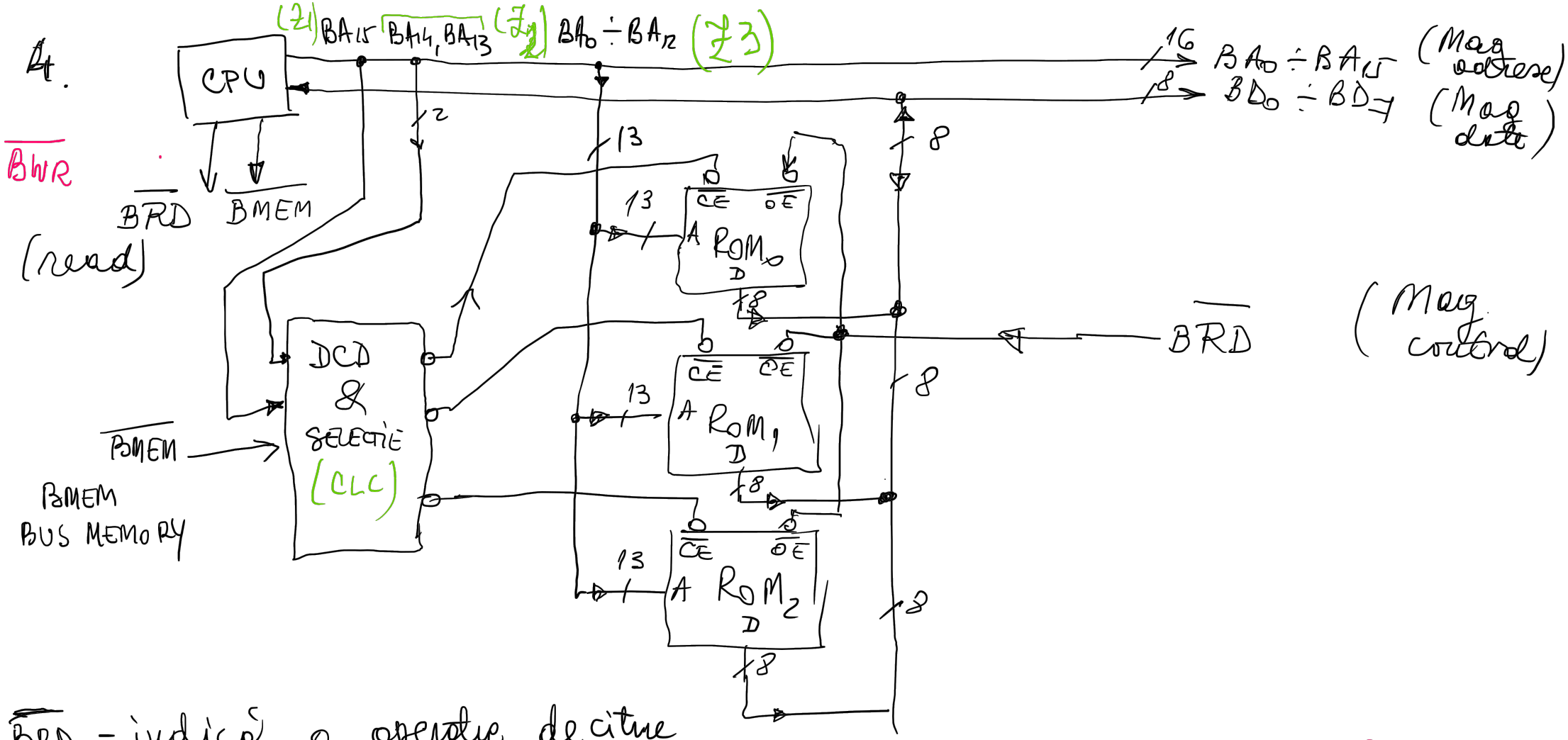
I – II – III

Zona I - linii neschimbate pe parcursul tabelului de adresare
Zona II - linii care identifică individual fiecare circuit de memorie (decodificare & selectare)
Zona III - linii care se schimbă funcție de adresa la fiecare circuit de memorie

OBS - Zona I se poate să nu existe.
Zonele II și III se pot suprapune partial sau total.


CPU
16
8
BA (bus address)
BD (bus data)
BC* (bus control)
8
8
8
ROM0
ROM1
ROM2
D0 ÷ D7
CE
(adr. compl.)
Z11
Z1
Bloc DCD & SELECTIE
CEROM0
CEROM1
CEROM2

4.

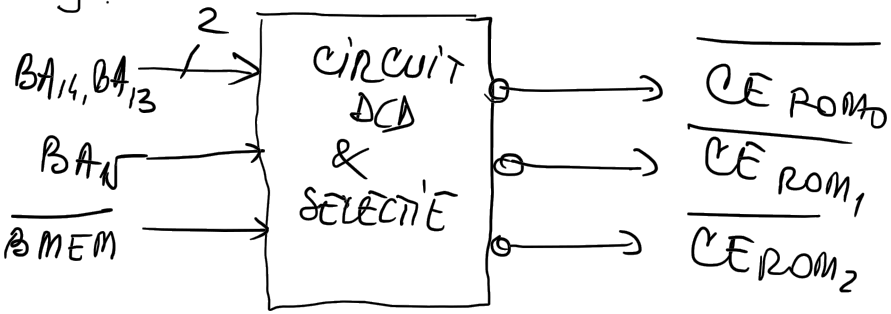


$\overline{BRD}$ - indică o operație de citire efectuată de CPU

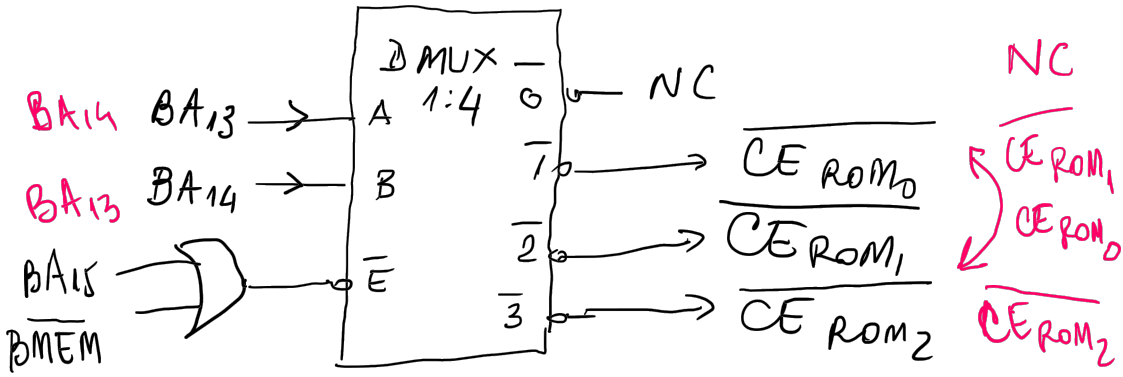
$\overline{BMEM}$ - indică lucrul cu memoria efectuat de CPU

$\overline{BWR}$ - indică operație de scriere

5.



(CLC)



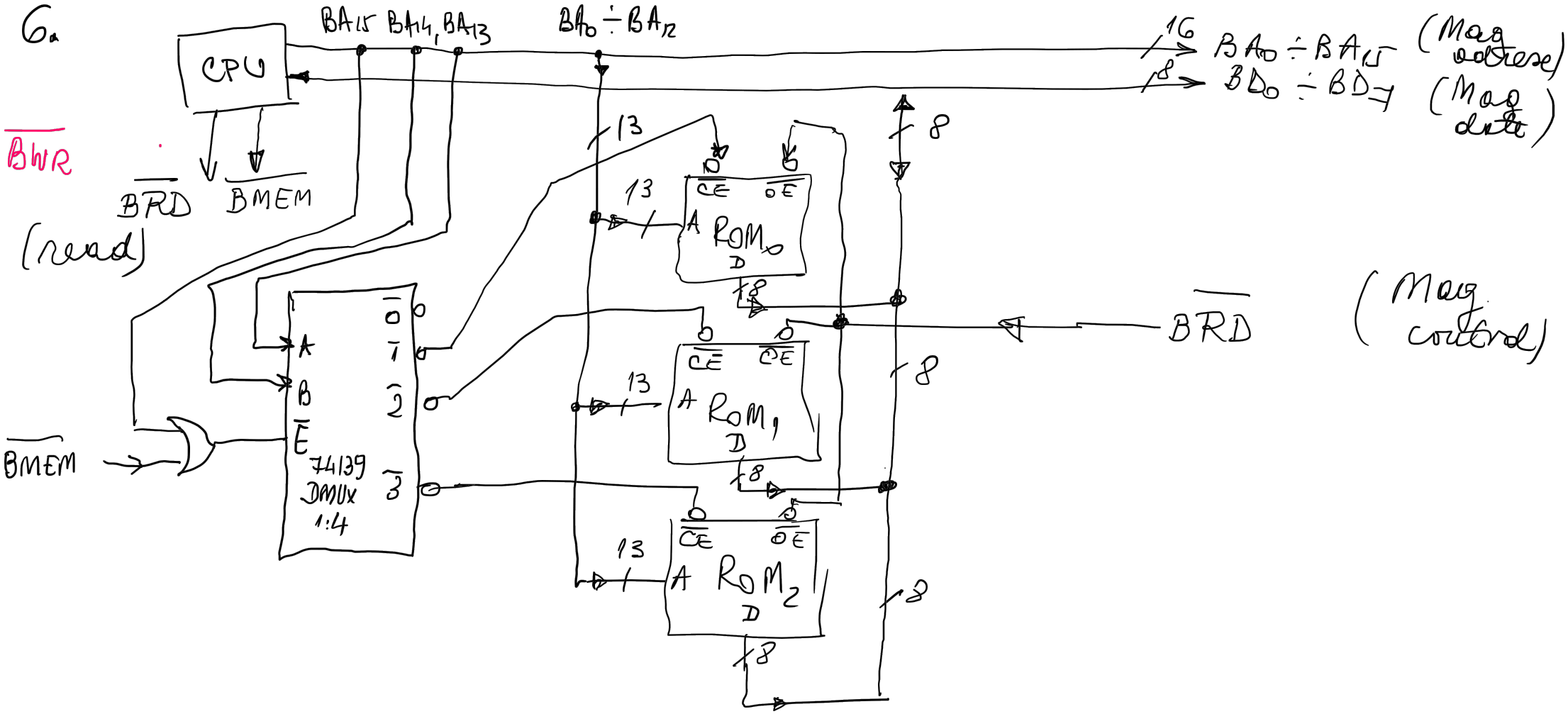
Proiectare CLC
- cu porti logice (U.K.) – SSI
- cu circuite MSI (DMUX, DCD, MUX) – Medium Scale Integration
- cu mem. ROM – LSI VLSI
- cu circuit PLA

* Proiectare cu DMUX
- nr ieșiri DMUX ≥ nr. semnale $\overline{CE}$ generate (3)
- nr. intrări selectie DMUX ≥ nr linii adrese din zona ii (2)
- se alege DMUX minimal care indeplineste ambele conditii

Tabel adresare

⇒ alegere DMUX 1:4

6.


CPU
BA15 BA14, BA13
BA0 ÷ BA12
16
BA0 ÷ BA15
(Mag address)
8
BD0 ÷ BD7
(Mag data)
BWR
BRD
BMEM
(read)
13
CE
OE
A
ROM0
D
8
BRD
(Mag control)
ROM1
ROM2
BMEM
A
B
E
74139
DMUX
1:4
0
1
2
3

Problema. In zona de memorie ROM astfel creata, la adresa 17767(10) a fost scrisa data 108(10). CPU citind data de la aceiasi adresa regaseste valoarea 172(10). Identificati circuitul care poate fi defect si liniile de date cu probleme.

Solutie

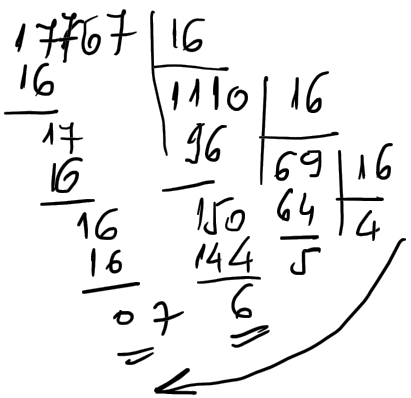
1. Transformarea în hexa a adresei
$17767_{(10)} = 4567_h \in ROM_1$

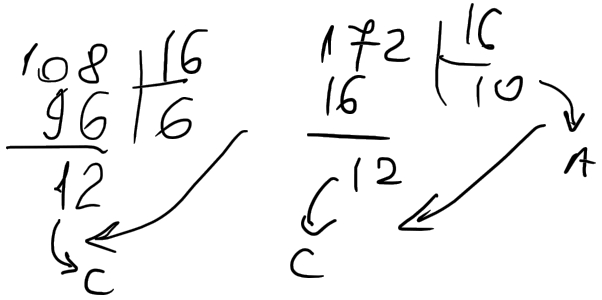
2. Transformarea în hexa a datelor

$108_{(10)} = 6C_h$
$172_{(10)} = AC_h$

~~DATA~~	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	~~DATA~~ ZECIMAL
6Ch	0	1	1	0	1	1	0	0	108(10)
ACh	1	0	1	0	1	1	0	0	172(10)

D_7, D_6 - au probleme





END 10 apr 2025

Problema. Intr-un sistem de calcul magistrala de adrese este de 16 biti, iar magistrala de date de 8 biti. In acest sistem de calcul trebuie conectata o zona de memorie SRAM de 48k x 8 (48KB), folosind circuite de memorie SRAM de tipul 62128 cu organizarea 16K x 8. Adresa de inceput a zonei de memorie SRAM este sa fie 3000h, zona de memorie SRAM este continua, iar adresarea este completa. Prezentati proiectarea conectarii circuitelor de memorie pentru indeplinirea cerintelor.

Metodologie de rezolvare:

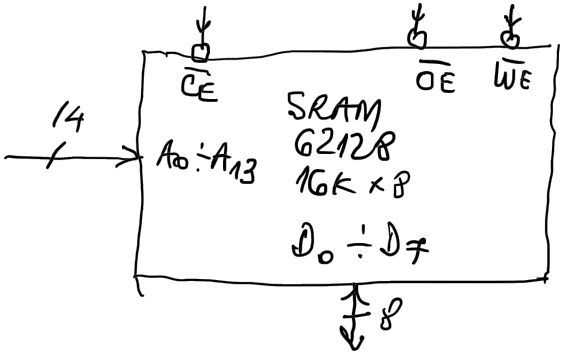
1. Prezentarea circuitului cu care se realizeaza implementarea.
2. Determinarea numarului de circuite pentru realizarea zonei de memorie.
3. Intocmirea tabelului de adresare a zonei de memorie.
4. Prezentarea schemei bloc a structurii ce trebuie realizata.
5. Proiectarea circuitului de decodificare si selectie
6. Conectarea circuitelor de memorie SRAM.

Solutii

1. SRAM 16K x 8

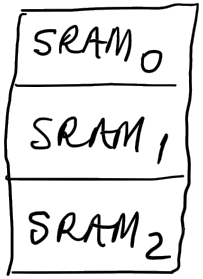
$16K = 2^4 \cdot 2^{10} = 2^{14}$ => 14 linii adrese $A_0 \div A_{13}$

8 linii de date $D_0 \div D_7$

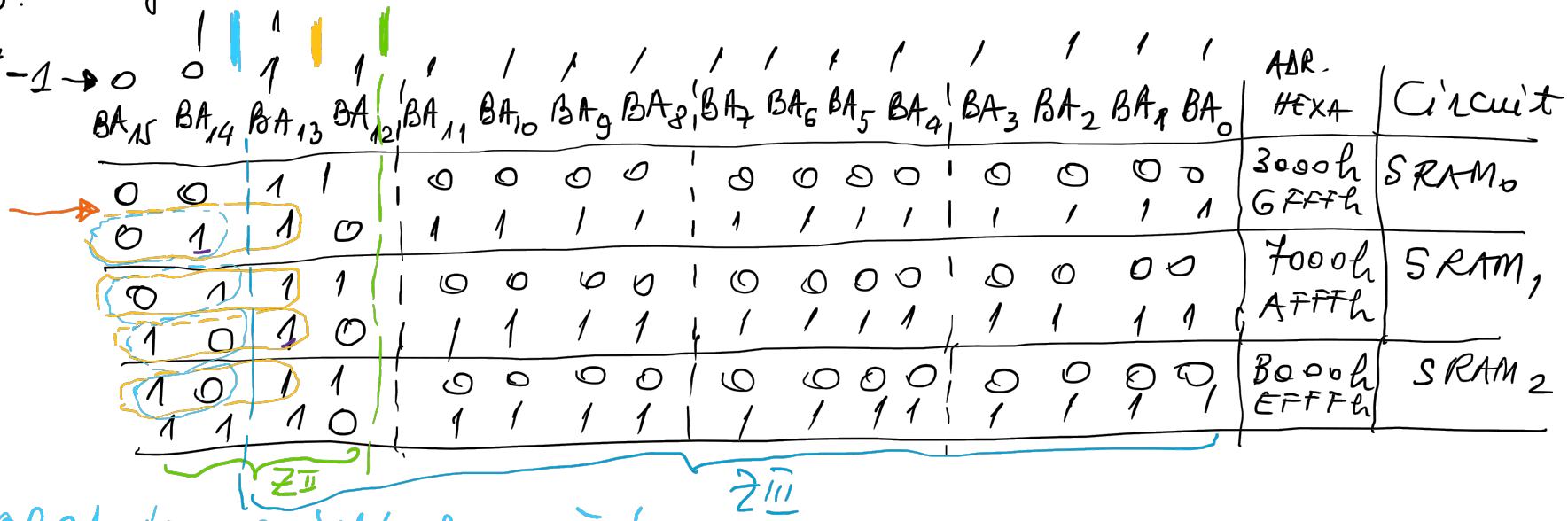


2. $\frac{48K \times 8}{16K \times 8} = 3 \times 1 = 3$ CI

3 linii, 1 coloane



3. Mag. de adrese are 16 biți $BA_{15} \div BA_0$



Obs1. Nu există Zona I !

Obs2. Sunt necesare 4 linii de adrese (BA_{15}, BA_{14}, BA_{13}, BA_{12}) pentru selectra individuală a celor 3 circuite SRAM _ (ZII)

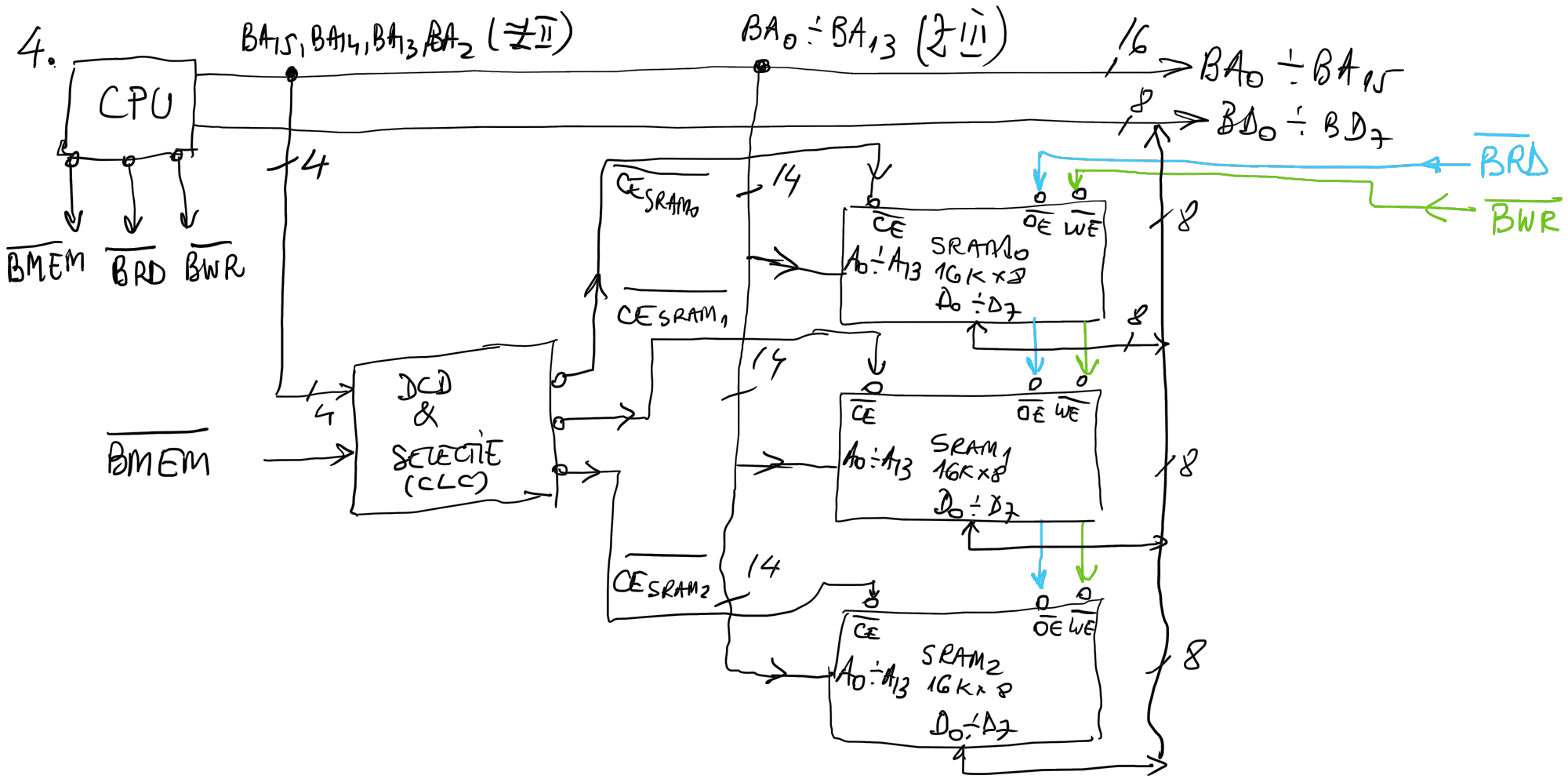
Fiecare circuit SRAM răspunde la câte 4 combinații ale celor 4 linii de adrese -

SRAM0 ← 0011, 0100, 0101, 0110 (pt $BA_{15} \div BA_{12}$)

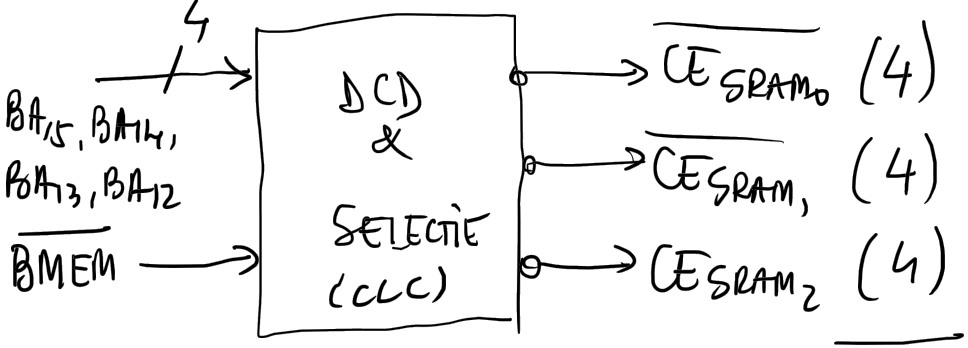
SRAM1 ← 0111, 1000, 1001, 1010

SRAM2 ← 1011, 1100, 1101, 1110

Obs3. Apare o suprapunere parțială între zonele ZII și ZIII pe linia BA_{13} BA_{12}


4.
CPU
BMEM
BRD
BWR
BA15, BA14, BA13, BA2 (ZII)
BA0 ÷ BA13 (ZIII)
16
BA0 ÷ BA15
8
BD0 ÷ BD7
BRD
BWR
4
DCD & SELECTIE (CLC)
BMEM
CE SRAM0
CE SRAM1
CE SRAM2
14
CE
OE
WE
A0 ÷ A13
SRAM0 16K x 8
SRAM1 16K x 8
SRAM2 16K x 8
D0 ÷ D7
8

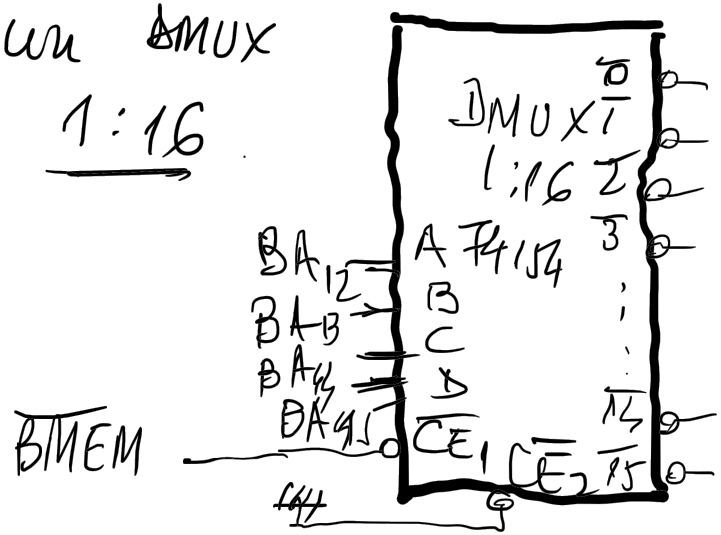
5.



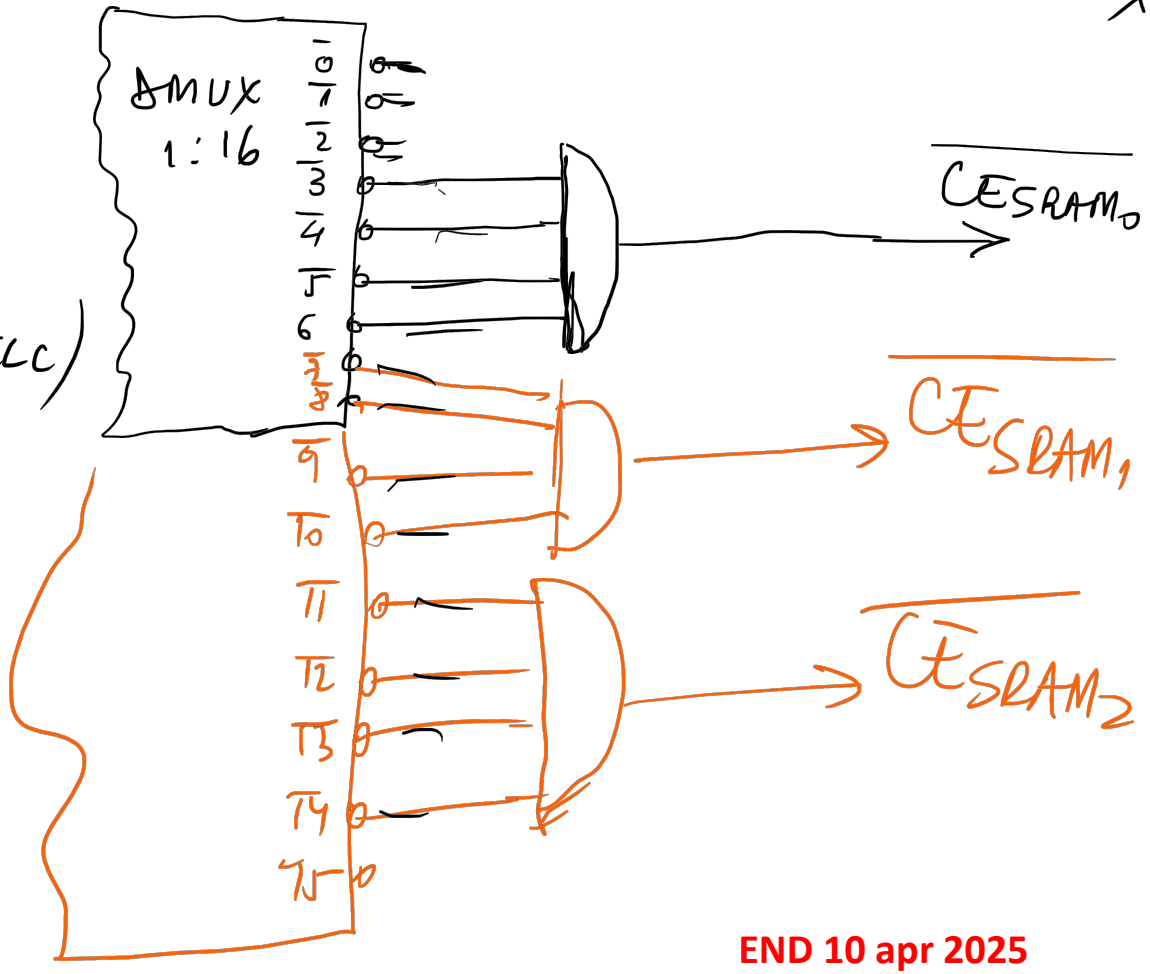
12!

Proiectăm cu DMUX
- intrări (Σ_{II}) – 4 (m - ieșiri reale din ccc)
- ieșiri – 12

Alegem un DMUX de tipul 1:16



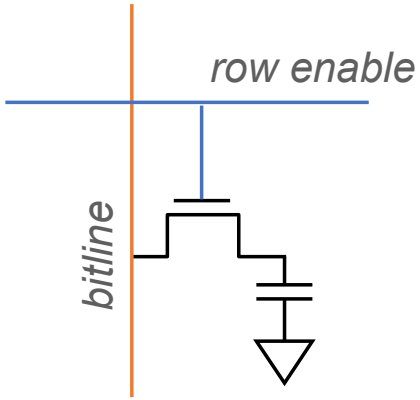
Obținerea unui semnal unic $\overline{CE_{SRAM_x}}$

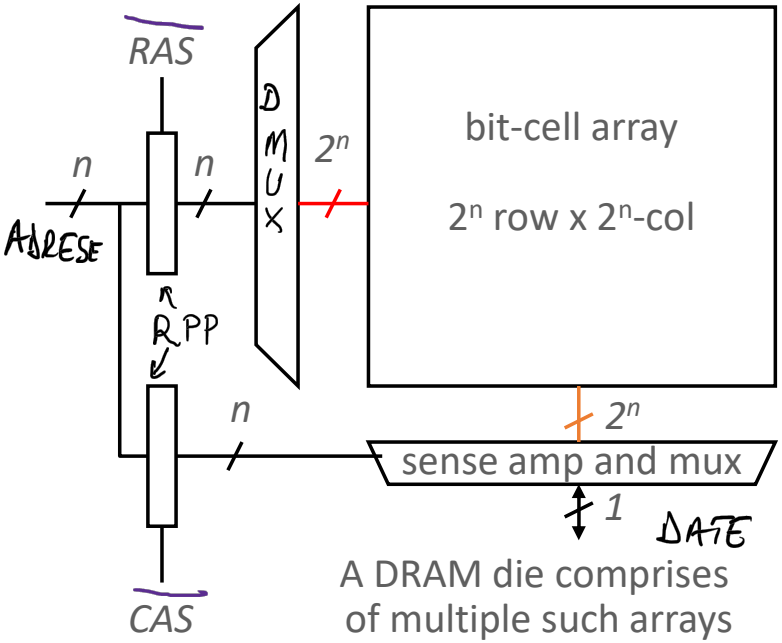


END 10 apr 2025

Memorii DRAM

- refresh



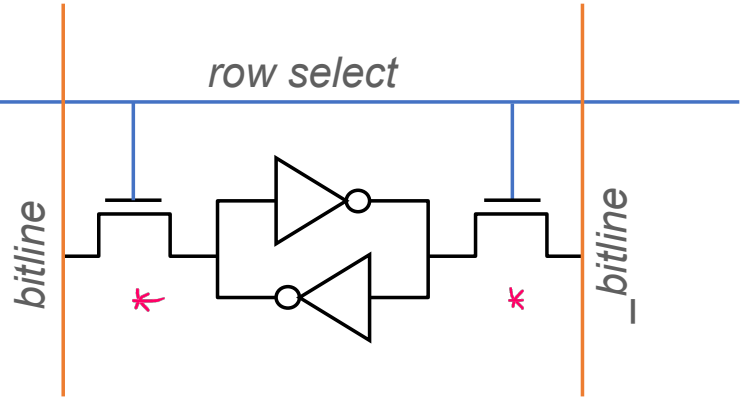


A DRAM die comprises of multiple such arrays

RAS - row address select

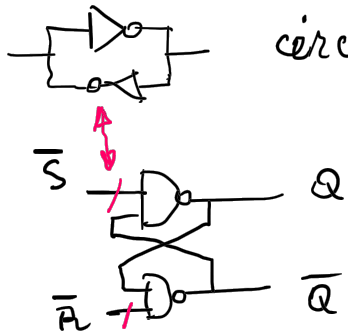
CAS - column address select

6 tranzistoare

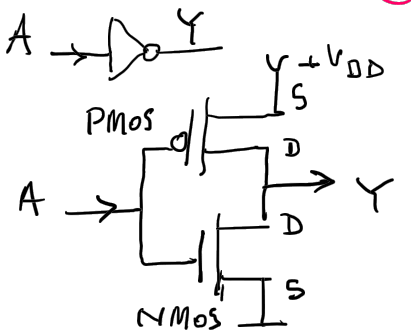


Celulă memorie SRAM

circuit regenerativ



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	1	1	0
1	0	0	1
1	1	memorare ($Q^{t+1} = Q^t$)	
0	0	nepermis	



CMOS

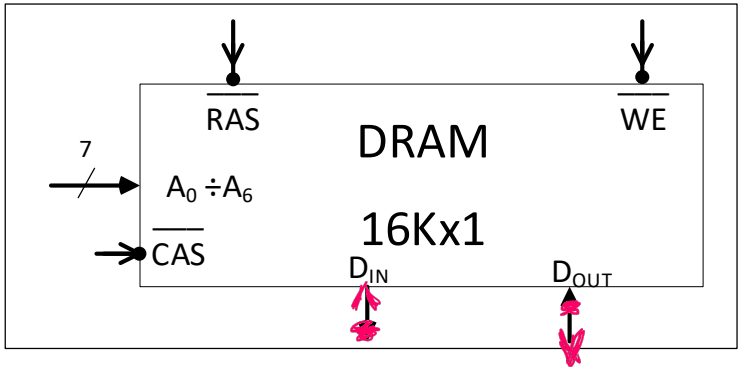
A=1 NMOS cond
Y=0

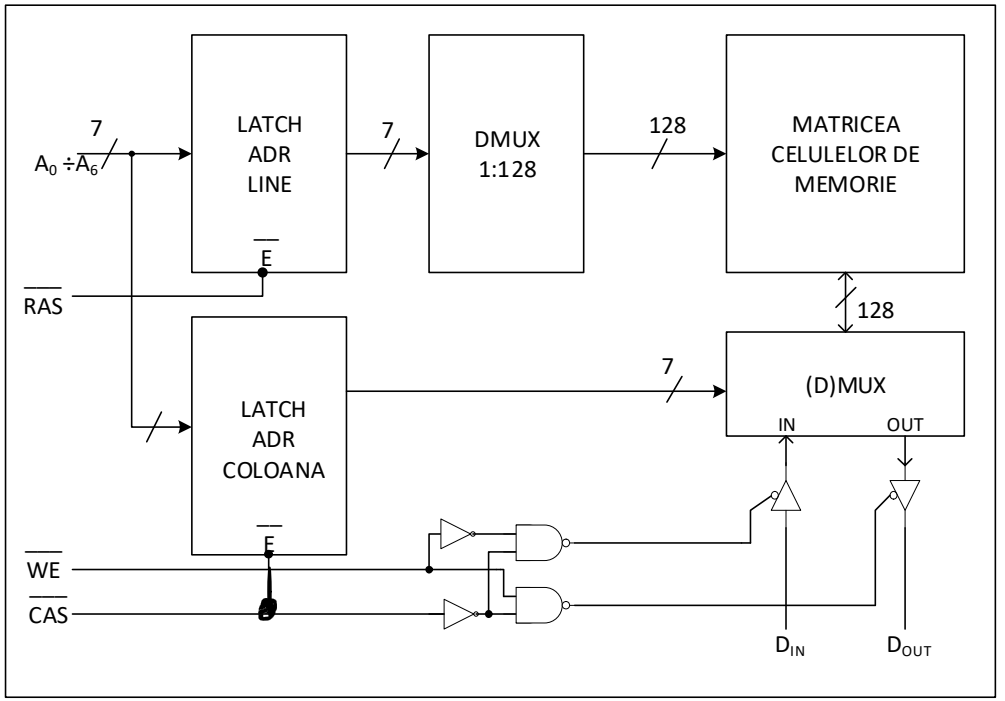
A=0 PMOS cond
Y=1

Structura unui circuit DRAM cu organizare 16k x 1 (circuitul 4116)

$16K = 2^4 \cdot 2^{10} = 2^{14} \Rightarrow$ 14 linii de adrese

DRAM $16K \times 1 \rightarrow \frac{14}{2} = 7$ linii de adrese





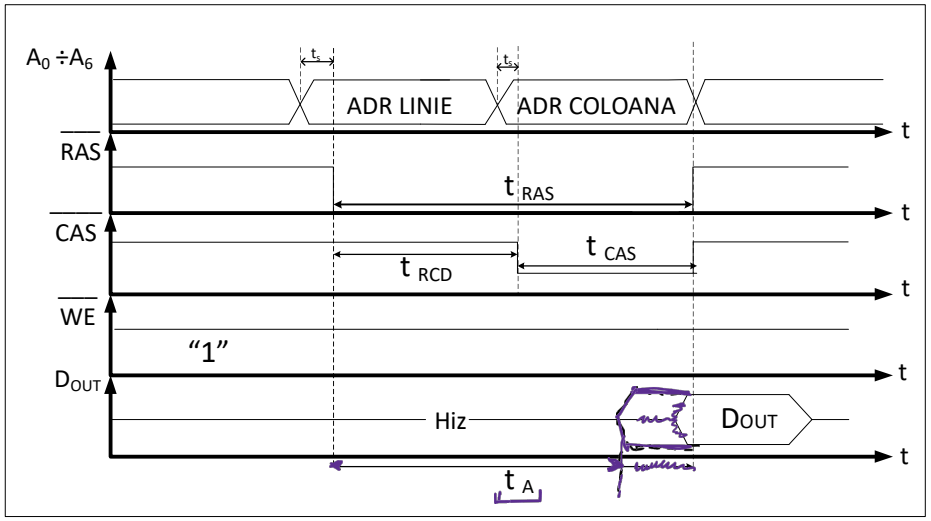
Caracteristici DRAM

- Existenta semnale RAS si CAS
- Numar linii de adrese injumatatit fata de numarul real (rezultat din organizarea circuitului)
- Linii de date separate intrare iesire
- Semnal scriere (WE)

RAS - Row Address Select

CAS - Column Address Select

Operatia de citire din DRAM



t_s - set-up $\geq 5ns$

$t_{RAS} \geq 70ns$

t_{RCD} - $\overline{RAS}$-$\overline{CAS}$-delay $\geq 20ns$

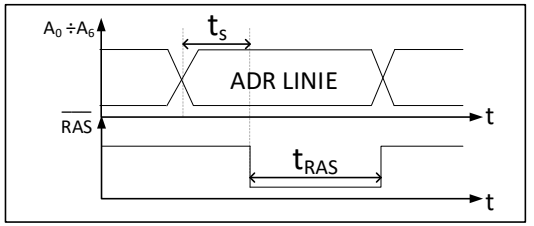
$t_{CAS} \geq 50ns$

t_A - access $\geq 60ns$ ← $t_A = 100ns$

1GHz → 1ns ÷ 10ns

1MHz → 1µs ÷ 10µs

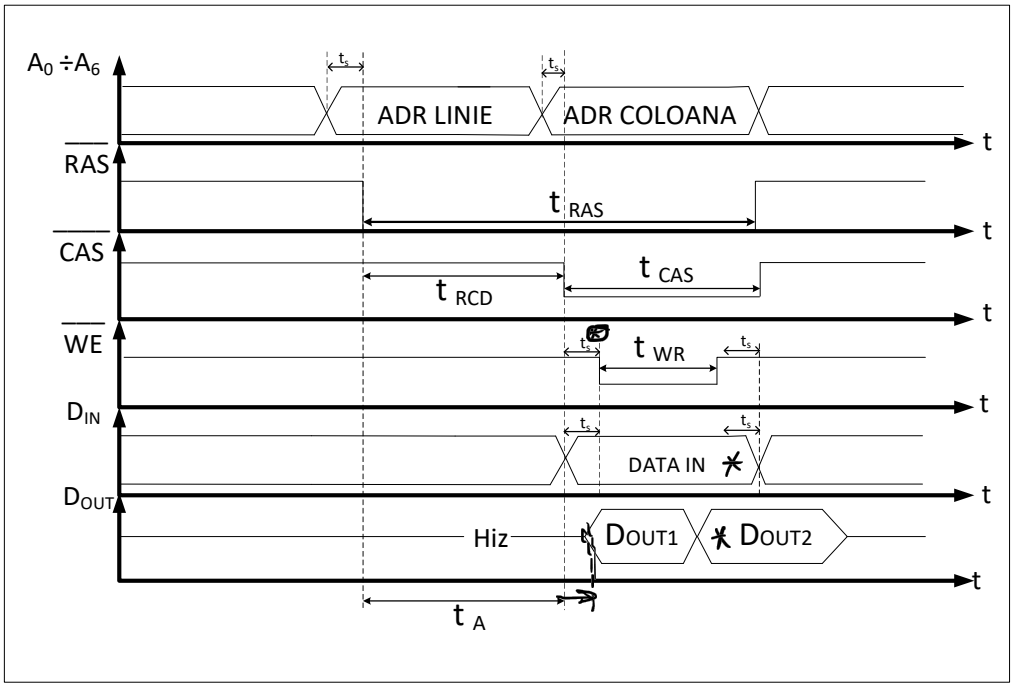
Operatia de refresh in DRAM



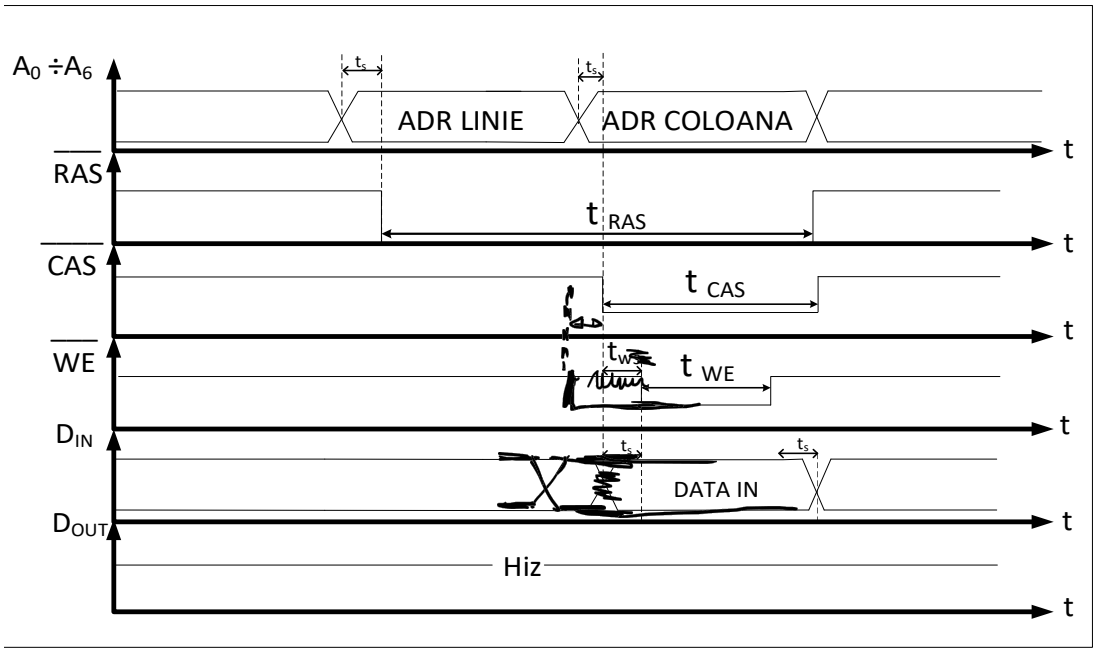
Clasificare DRAM

- DRAM asincron (fără CLK)
- DRAM sincron (cu CLK)
  - SDRAM (single rate DRAM) — CLK
  - DDRAM (double rate DRAM)
    - DDRAM
    - DDRAM2
    - DDRAM3
    - DDRAM6
    - (GDRAM)

Operatia Read-Write-Modified in DRAM (RWM)



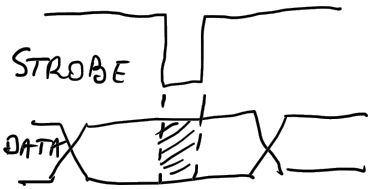
Operatia Early-Write in DRAM (EW)

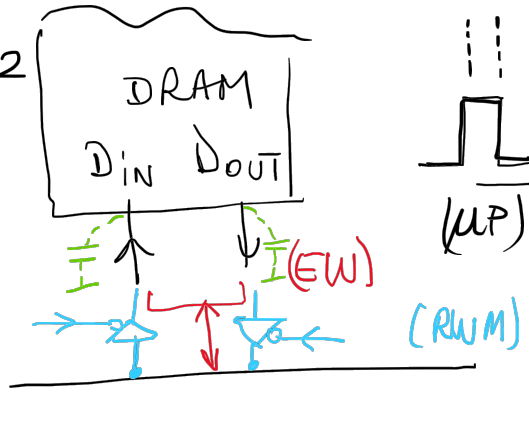


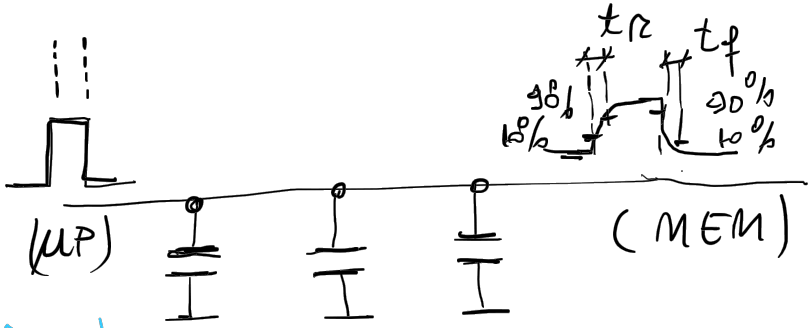
t_{CWD} – CAS – WR – delay

t_s set-up

DATA$_{IN}$ ≡ DOUT2



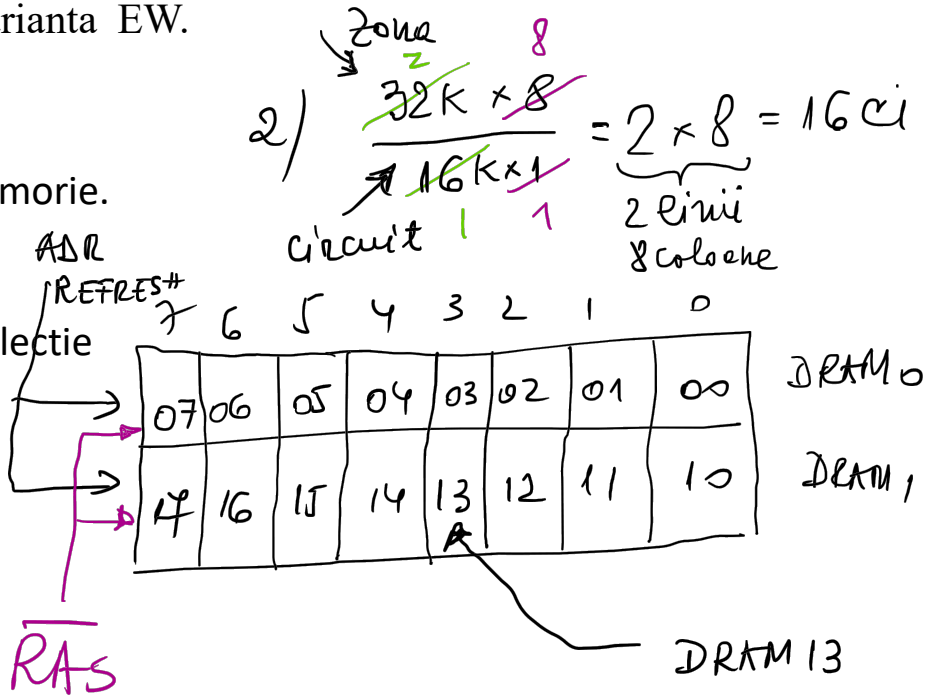




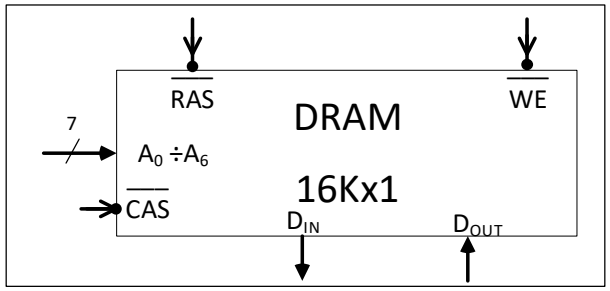
Problema. Intr-un sistem de calcul in care magistrala de adrese este pe 16 linii BA_0-BA_{15} si magistrala de date este BD_0-BD_7 se conecteaza o zona de memoria DRAM, **32k x 8**, folosind circuitele DRAM 4116, avand organizarea **16k x 1.** Zona de memorie incepe de la adresa 1000h, este continua si tehnica de adresare a ei este completa. Prezentati solutia hardware care corespunde enuntului, atat in varianta RWM, cat si in varianta EW.

Metodologie de rezolvare:

1. Prezentarea circuitului cu care se realizeaza implementarea.
2. Determinarea numarului de circuite pentru realizarea zonei de memorie.
3. Intocmirea tabelului de adresare a zonei de memorie.
4. Prezentarea schemei bloc a structurii ce trebuie realizata.
5. Proiectarea circuitelor de multiplexare, respectiv decodificare si selectie
6. Conectarea circuitelor de memorie.



1) $16K = 2^4 \cdot 2^{10} = 2^{14}$; 14 – nr linii adrese (circuite normale)

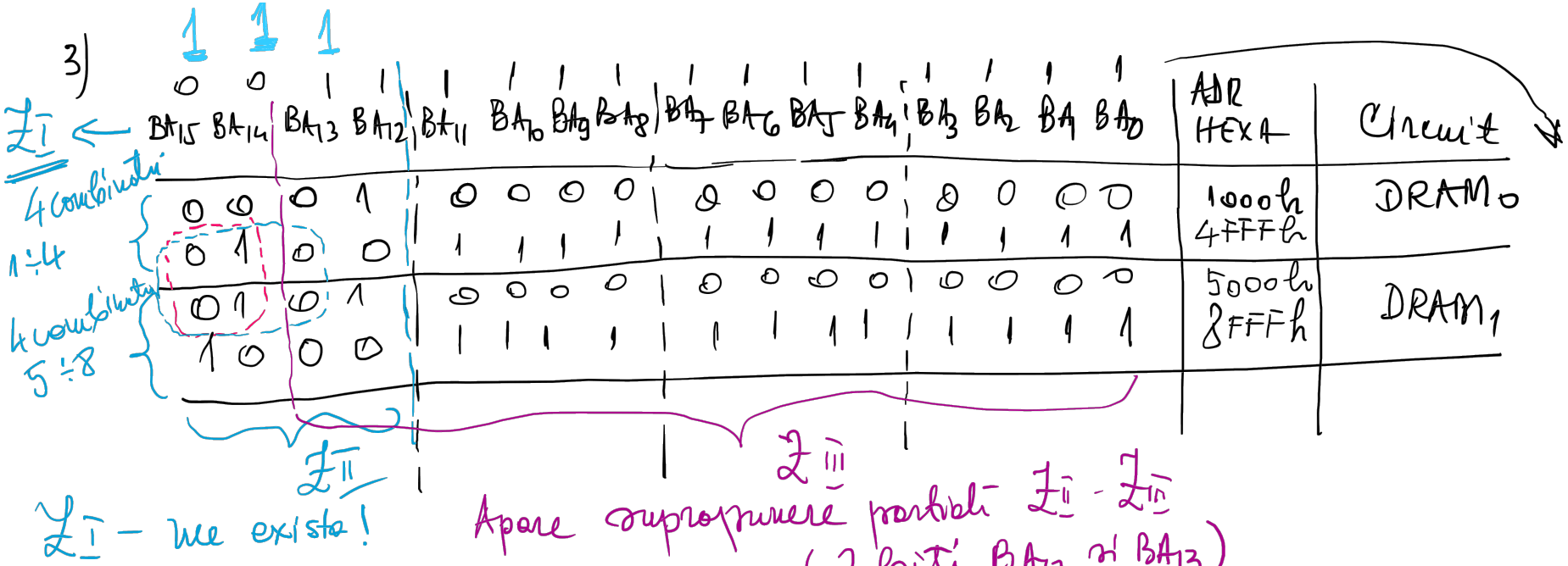


La DRAM: (ROM ; SRAM)

nr. linii adrese $\frac{14}{2} = 7$!

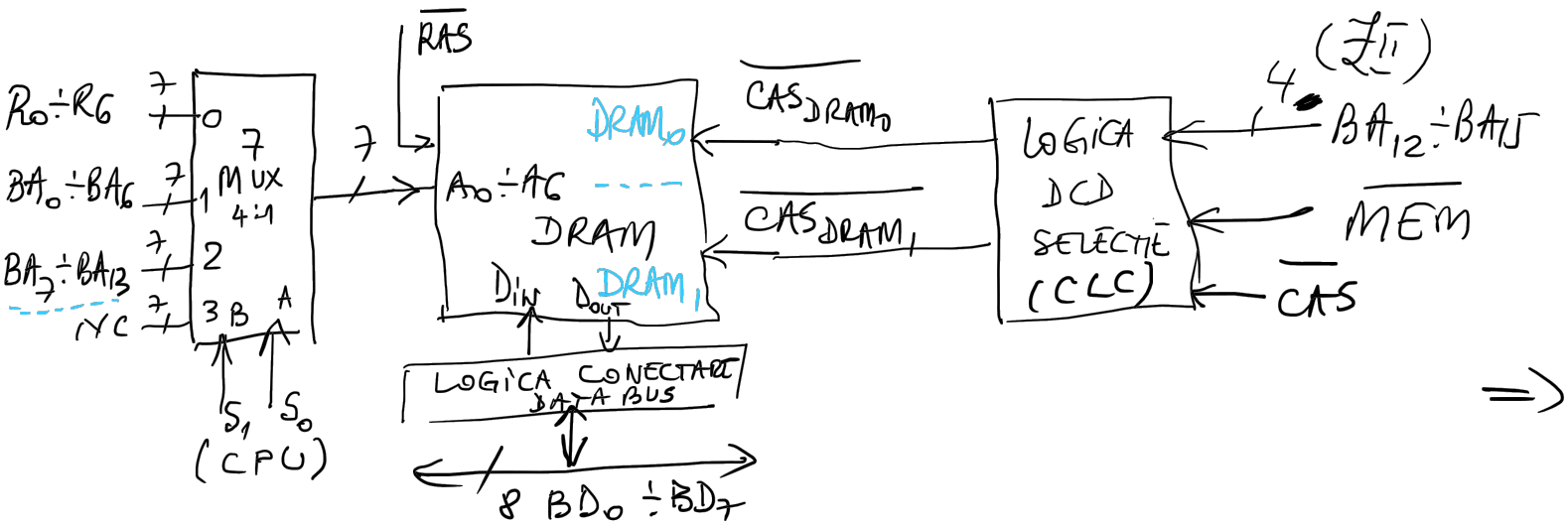
$A_0 \div A_6$!!

3)



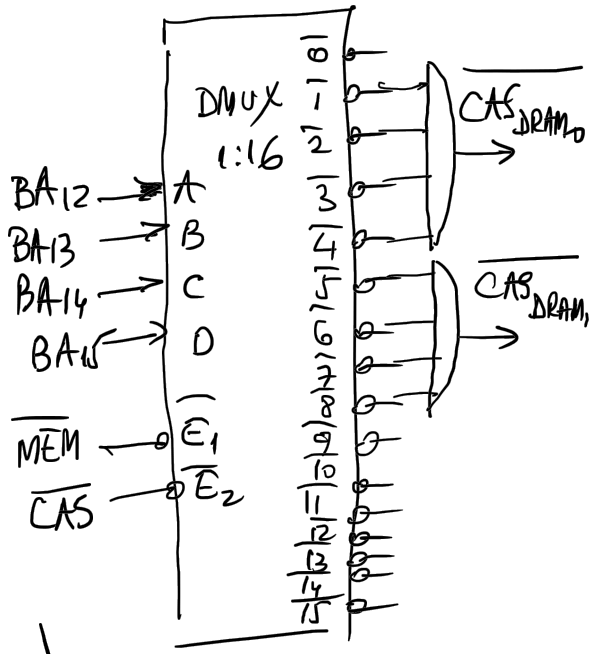
4) - Necesitate refresh!
- operație prioritară față de operarea CPU!
⇒ necesită refresh simultan la toate circ. DRAM (pt timp de disponibilitate maximizat la operarea CPU)
⇒ adresă de refresh (linie) unică pt toate circuitele DRAM
Și un semnal comun $\overline{RAS}$
- vor fi $2^7 = 128$ adrese de refresh (128 operații în cca 2ms)

- adresă de refresh comună (adr linie comună) la toate cele 2 x 8 = 16 circuite DRAM
- există un generator pentru adresele de refresh (numărător $2^7 = 128$ stări)
$R_0 \div R_6$ ⇒ $A_0 \div A_6$ (DRAM)
- CPU aplică adresa la circ. DRAM
$BA_0 \div BA_6$ (adresă linie) însoțită de $\overline{RAS}$
$BA_7 \div BA_{13}$ (adresă de coloană) însoțită de $\overline{CAS}$

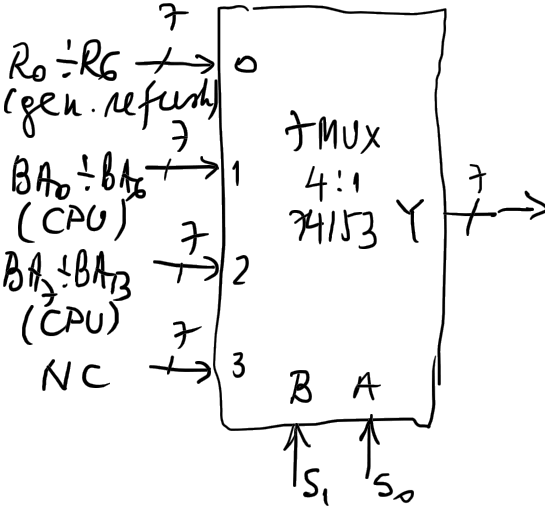


⑤b) Logica DCD & SEL
- implementare CLC cu DMUX
* 4 intrări selecte
* $2 \times 4 = 8$ ieșiri

$\Rightarrow$ DMUX 1:16 (4 linii sel)



⑤ a) Circuit multiplexare adrese și refresh



la $A_0 \div A_6$ de la toate circ. DRAM

CPU acționează astfel:

$S_1 S_0 = 00$ se aplică adresa de refresh ($R_0 \div R_6$) + $\overline{RAS}$

$S_1 S_0 = 01$ se aplică adresa de linie ($BA_0 \div BA_6$) + $\overline{RAS}$

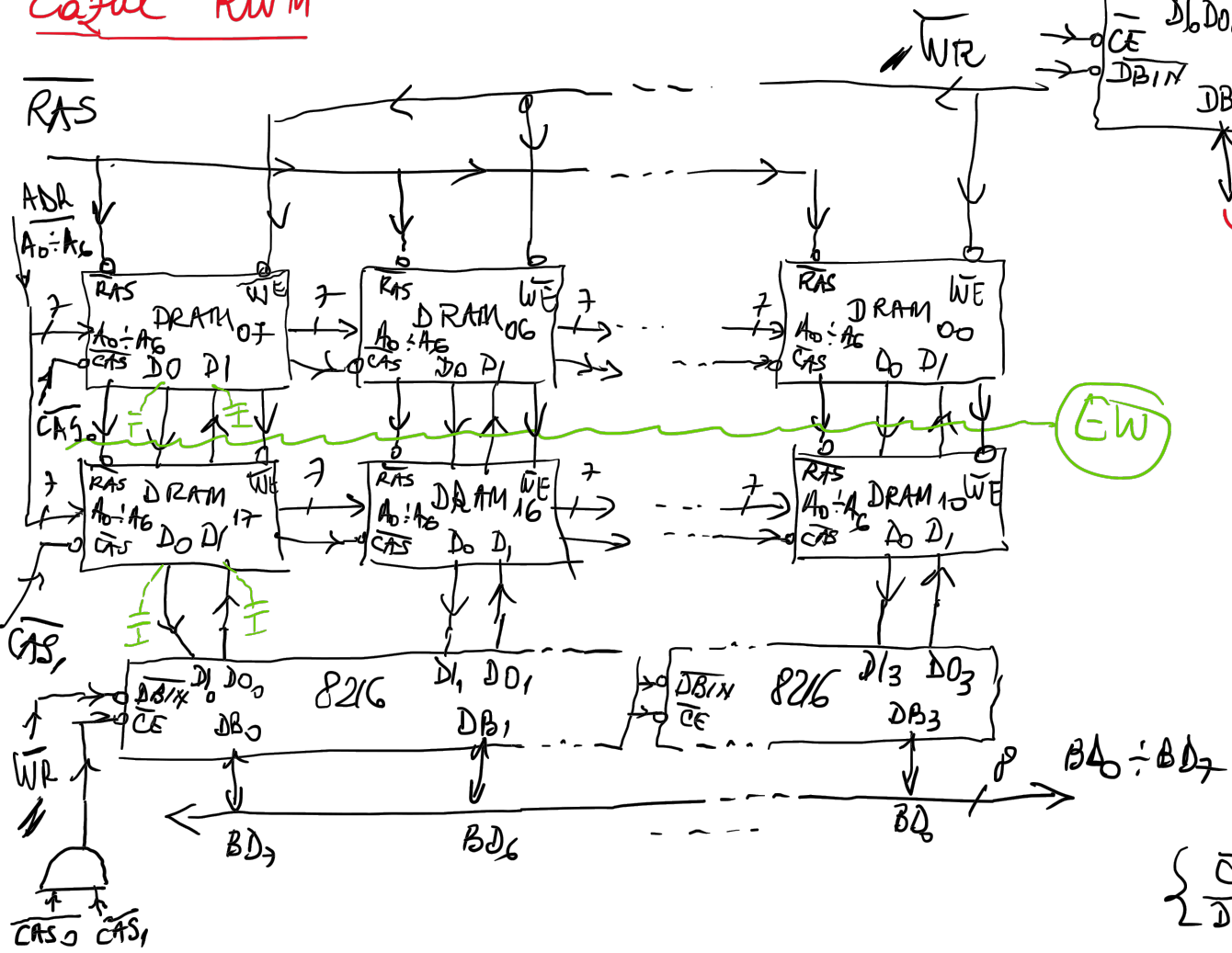
$S_1 S_0 = 10$ se aplică adresa de coloană ($BA_7 \div BA_{13}$) + $\overline{CAS}$

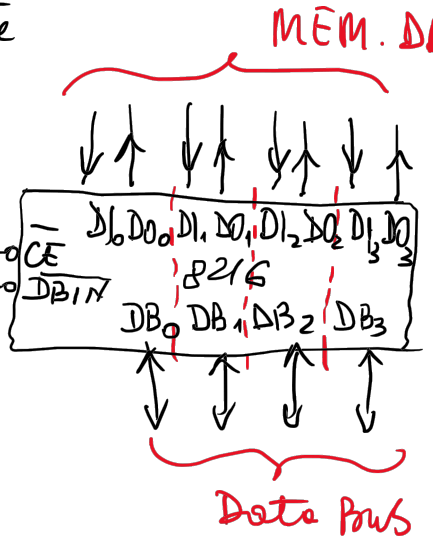
5c) Conectarea la magistrala de date

RWM - Read Write Modified

EW - Early Write

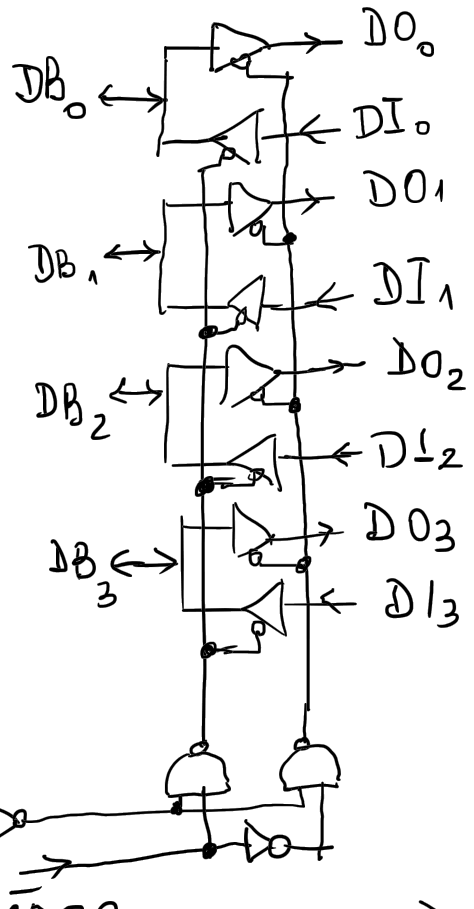
Cazul RWM





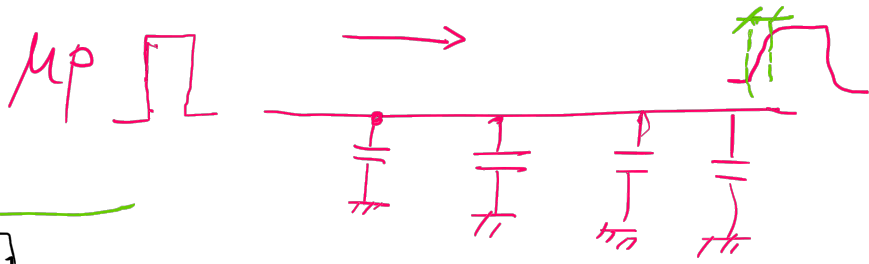
Intel 8216

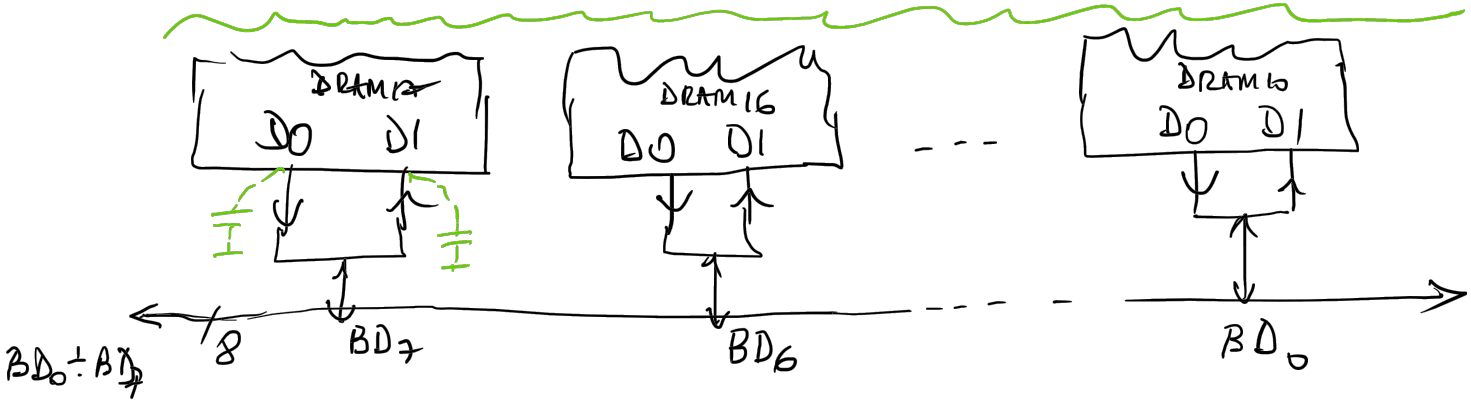
4 tampoane bidirectionale cu TS



$\begin{cases} \overline{CE}=0 \\ DBIN=1 \end{cases} \Rightarrow DI_i \rightarrow DB_i \quad \begin{cases} \overline{CE}=0 \\ DBIN=0 \end{cases} \Rightarrow DB_i \rightarrow DO_i$

Cazul EW





OBS

1. Semnalele $\overline{RAS}$, $\overline{CAS}$, S_1, S_0 – nu sunt generate direct de CPU. CPU emite alte semnale din care proiectantul HW formează semnalele menționate.

2. La RWM – semnalele $\overline{CAS_i}$ ($\overline{CAS}$) trebuie să apară înaintea semnalului $\overline{WR}$
   La EW – semnalul $\overline{WR}$ trebuie să apară înaintea $\overline{CAS}$, ($\overline{CAS}$)

Tot proiectantul HW va genera semnalele în ordinea menționată plecând de la alte semnale emise de CPU.

Problema În zona de memorie DRAM proiectată anterior se înscrie la adresa $23456_{(10)}$ data $151_{(10)}$ ulterior citind de la aceeași adresă se regăsește data $215_{(10)}$ - (fără să se mai scrie intretimp o altă dată la adresa respectivă).
Identificați circ. defecte.

Sol.

1) $23456_{(10)} = 5BA0\,h \Rightarrow$ DRAM

2) $151_{(10)} = 97h$
$215_{(10)} = D7h$
$\Rightarrow$ DRAM 16

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	DATA HEX
1	0	0	1	0	1	1	1	97h
1	1	0	1	0	1	1	1	D7h

151 | 16
144 | 9
7

215 | 16
16 | 13
55
48
7

23456 | 16
16 | 1466 | 16
74 | 144 | 91 | 16
64 | 26 | 80 | 5
105 | 16 | 11
96 | 10
=96
96
0

Memoria CACHE

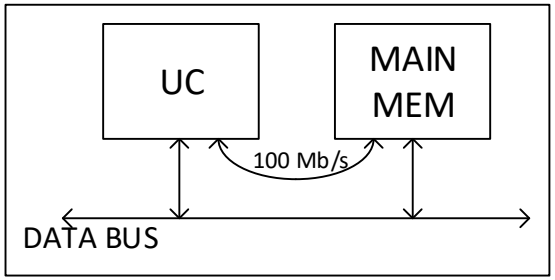
Este o zona de memorie SRAM care este plasata intre procesor si memoria principala de tip DRAM; este inaccesibila utilizatorilor.

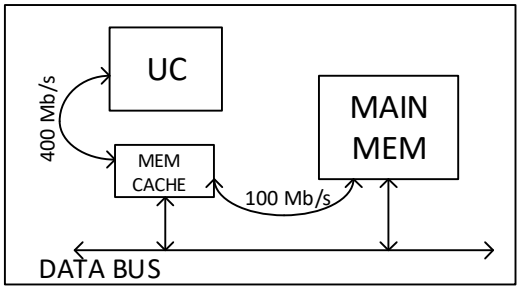
In fapt este un mecanism care creste viteza de lucru al respectivului sistem de calcul si se bazeaza pe urmatoarele principii si considerente:

- Un prim principiu este cel al localizarii temporale care afirma ca daca la un moment de timp, procesorul opereaza cu o zona de locatii din memoria principala este probabil ca dupa o perioada de timp procesorul va opera cu aceleasi locatii de memorie.

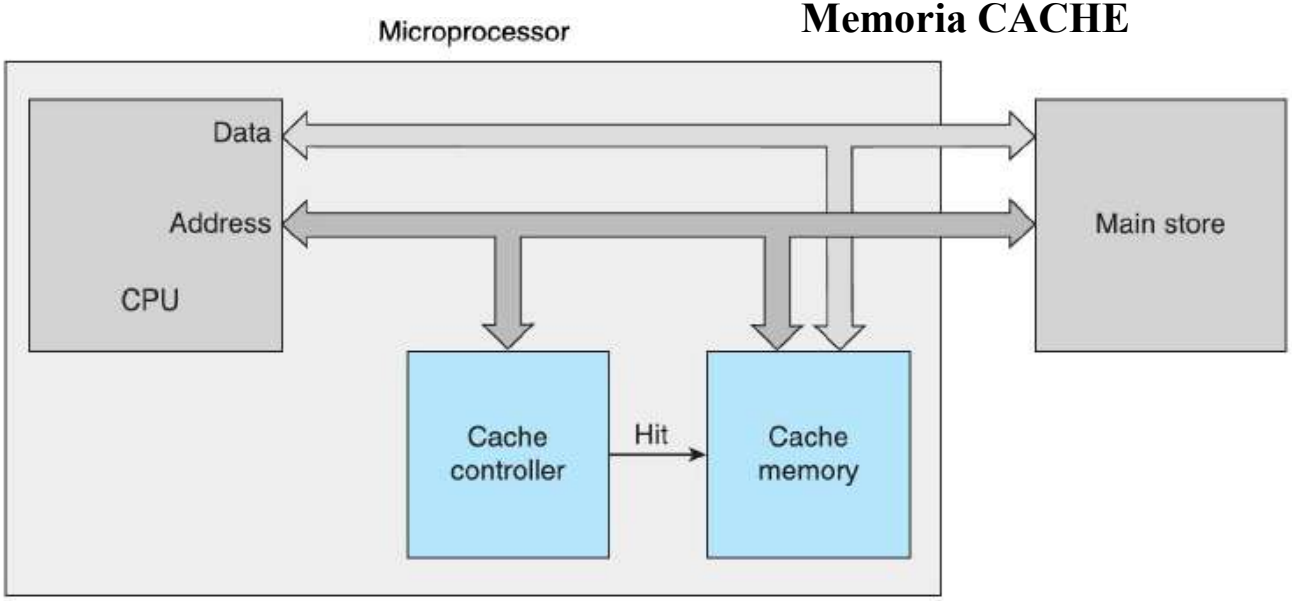
- Un al doilea principiu este cel al localizarii spatiale care afirma ca daca procesorul opereaza cu locatii dintr-o zona a memoriei principale este probabil ca el va opera si cu locatii apropiate respectivei zone de memorie.

- Un al treilea principiu este denumit 10/90 care afirma ca 90% din timpul de rulare al unui procesor, el evolueaza in 10% din memorie.





Memoria CACHE

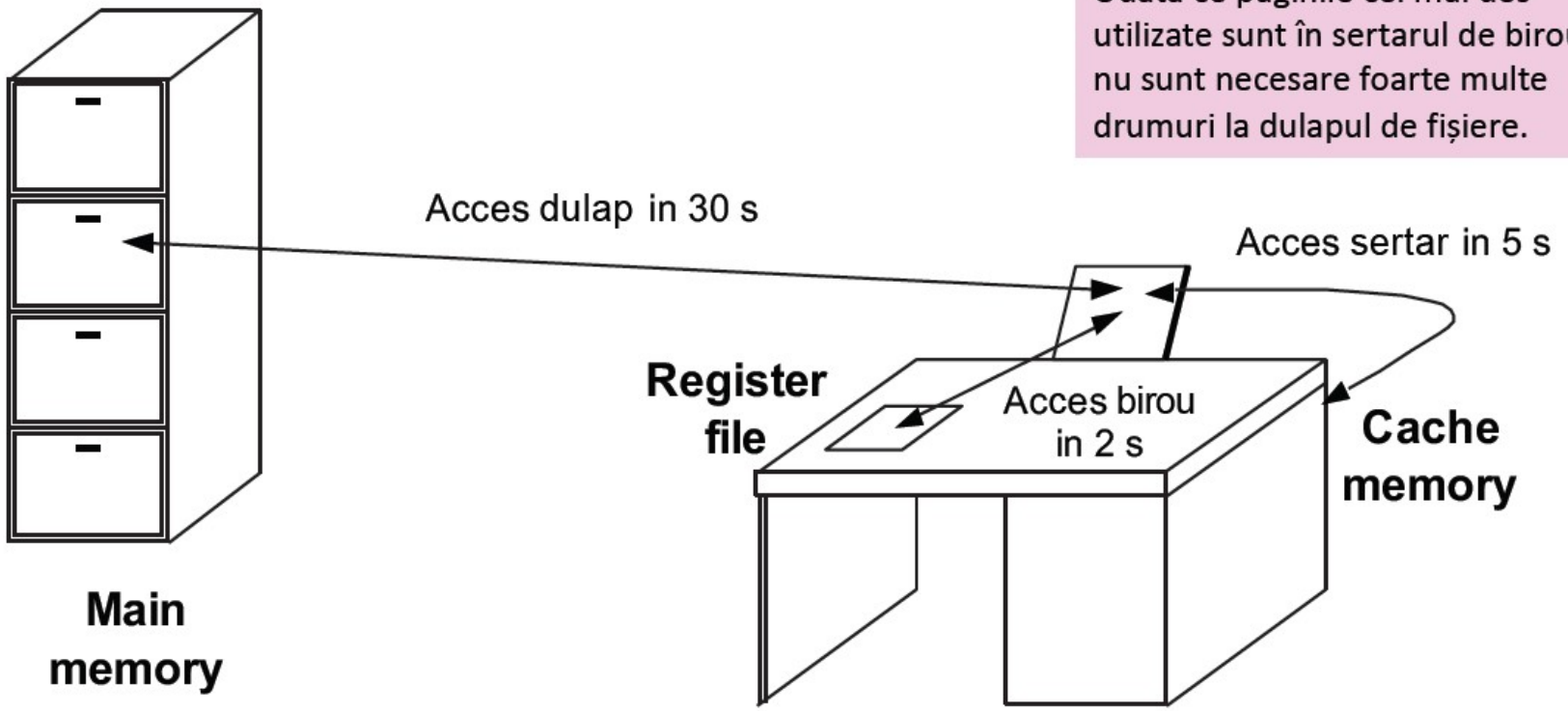


Politici de inlocuire a informatiilor din cache:

LRU – least recently used;
LFU – least frequently used;
FIFO – first in first out;
Random – aleatoriu.

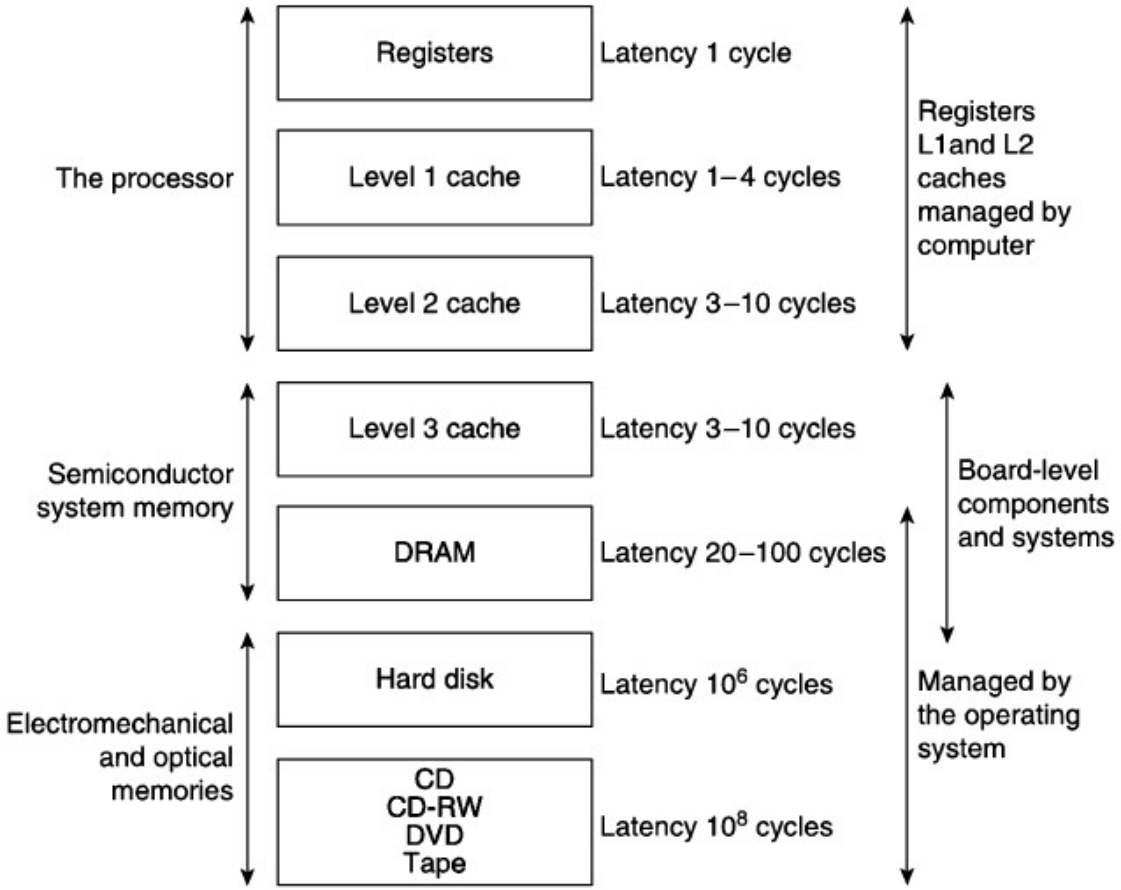
Analogie de funcționare pentru memoria cache

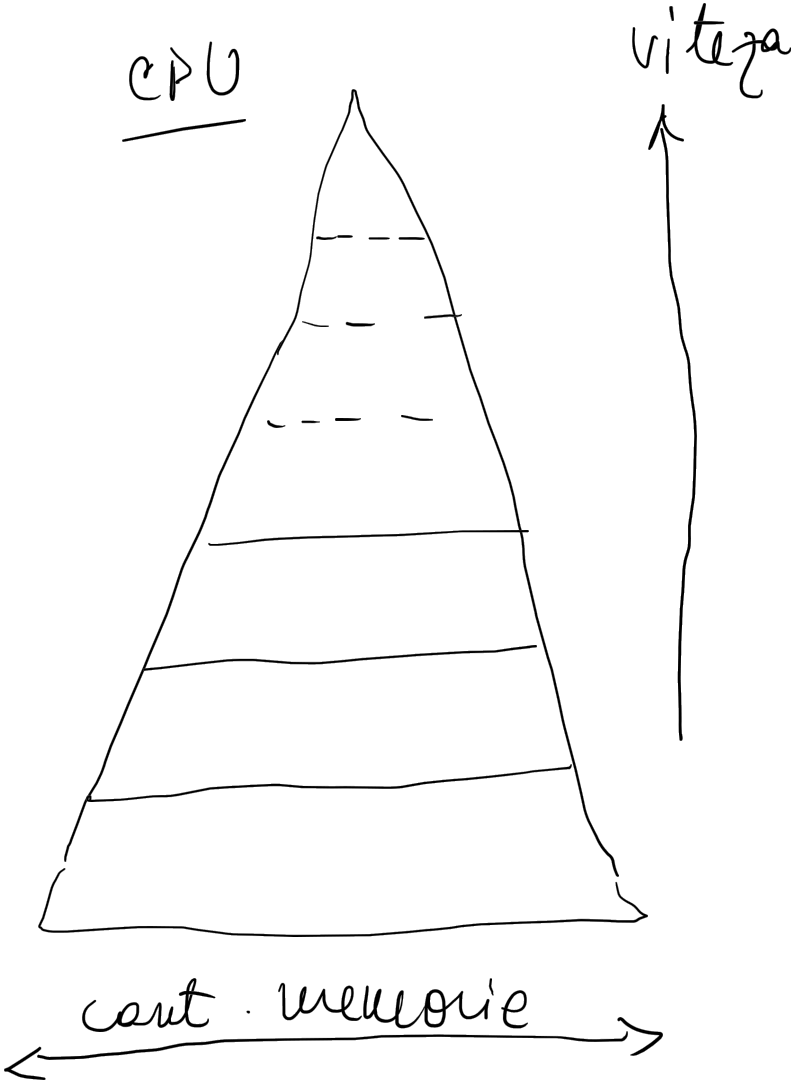
Odată ce paginile cel mai des utilizate sunt în sertarul de birou, nu sunt necesare foarte multe drumuri la dulapul de fișiere.



Foile de pe birou (registre) sau dintr-un sertar (cache) sunt mult mai ușor de accesat decât cele dintr-un dulap de fișiere (memoria principală).

Ierarhia memoriei in sistemele de calcul





Memoria CACHE

Ideea mecanismului memoriei cache este acela ca in aceasta memorie se face o copie a zonei parcurse de procesor din memoria principala. Cand procesorul incearca sa acceseze o locatie de memorie, o prima cautare a ei se face in memoria cache, si daca informatia este gasita, atunci avem un **hit**, iar datele sunt extrase din cache fara a se mai apela memoria principala. Daca datele nu sunt gasite, atunci avem un **miss** si cautarea are loc mai departe in memoria principala.

Memoria cache este gestionata de un mecanism de comanda si control care are mai multe functionalitati:

- Cautarea locatiei dorite de procesor din memoria principala in memoria cache si declararea rezultatului acestei operatii prin HIT sau MISS;
- Actualizarea continutului memoriei cache; acest lucru presupune identificarea locatiilor cache-ului (denumite SLOT-uri) care nu sunt folosite de procesor si umplerea acestora cu noi continuturi de locatii din memoria principala in care procesorul evolueaza.

Exista 3 topologii de organizare ale memoriei cache:

- FULLY ASSOCIATIVE (total asociativa)
- DIRECT MEMORY MAPPED (cu translatare directa in memorie)
- N-WAYS SET ASSOCIATIVE (asociativa, avand n submultimi - cai)

Tipuri de cache miss

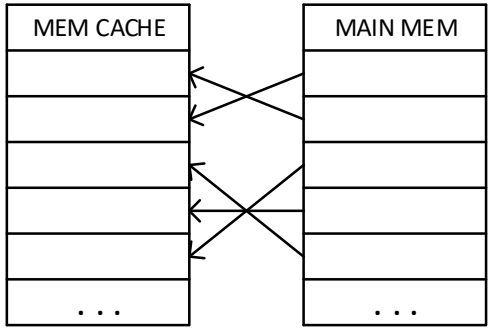
Compulsory: prima referință la o linie (cold start misses)
– *Miss-uri care se petrec și pentru un cache de dimensiuni infinite*

Capacity: cache prea mic pentru a ține toate datele necesare unui program
– *Miss-uri care se petrec și dacă avem o politică perfectă de înlocuire a liniilor din cache*

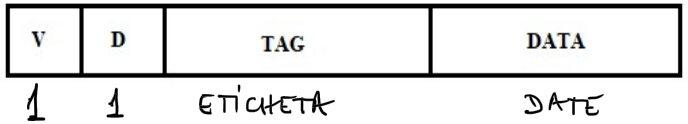
Conflict: miss-uri care se petrec datorită coliziunilor datorate strategiei de amplasare a liniilor
– *Miss-uri care nu ar apare dacă am avea o asociativitate completă*

Memoria cache organizata FULLY ASSOCIATIVE – total asociativ

Denumirea organizarii provine de la modul in care continutul memoriei principale este translatat (mapat) in memoria cache. Un desen care explica tehnica de translatare este urmatorul:



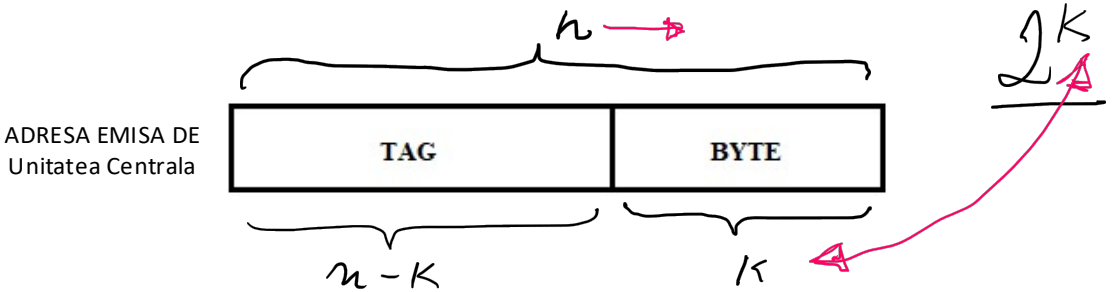
Locatiile din memoria principala se translateaza in SLOTURILE cache-ului in mod liber.
In acest mod de organizare un slot al cache-ului are urmatorul format:

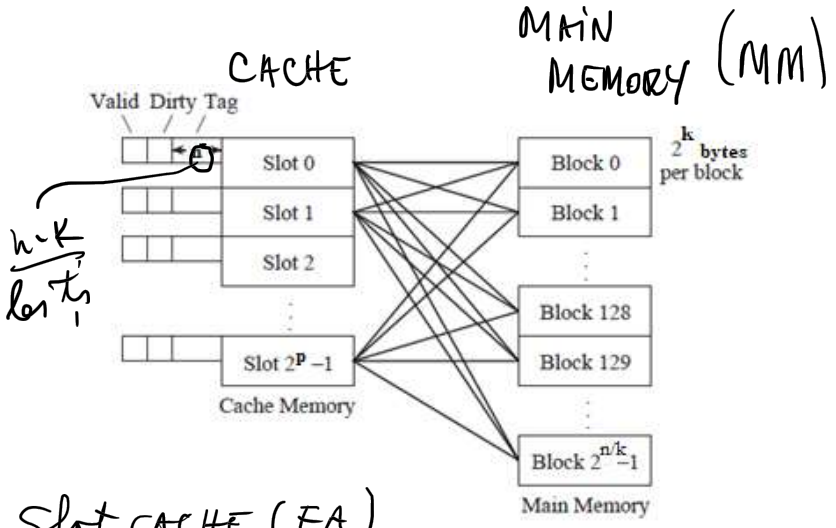


- V – provine din VALID, indica daca informatia continuta in respectivul slot al cache-ului este utilizabila (similara sau nu cu informatia din memoria principala), astfel pe informatia obtinuta procesorul nu se poate baza, iar continutul slotului trebuie inlocuit.
- D – DIRTY, indica daca informatia din slotul respectiv a fost modificata; se foloseste pentru modificarea memoriei.
- TAG – eticheta, format dintr-un numar de biti care exprima o parte a adresei din memoria principala la care se gasesc datele care isi au copia in slotul respectiv al cache-ului. Prin intermediul bitilor din campul TAG se va putea face recunoasterea informatiei pe care procesorul asteapta sa le preia din cadrul memorie cache.

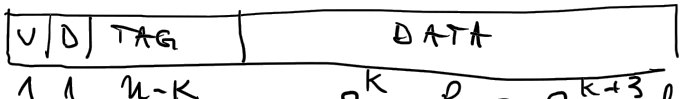
- DATA – continutul informatiei propriu-zise pe care procesorul o poate accesa in situatia hiturilor, beneficiindu-se de timpul de acces redus al respectivelor circuite. Particularitatea campului DATA dintr-un slot al cache-ului este aceea ca in el sunt continuturile mai multor locatii succesive din memoria principala. Numarul acestor locatii succesive este o putere intreaga a lui 2.

Din punct de vedere al unei adrese emise de unitatea de control pentru accesarea unei locatii din memoria principala, impartirea bitilor de adrese cere urmatorul format:





Slot CACHE (FA)



1 + 1 + (n-K) + 2^{K+3} biti în slot cache

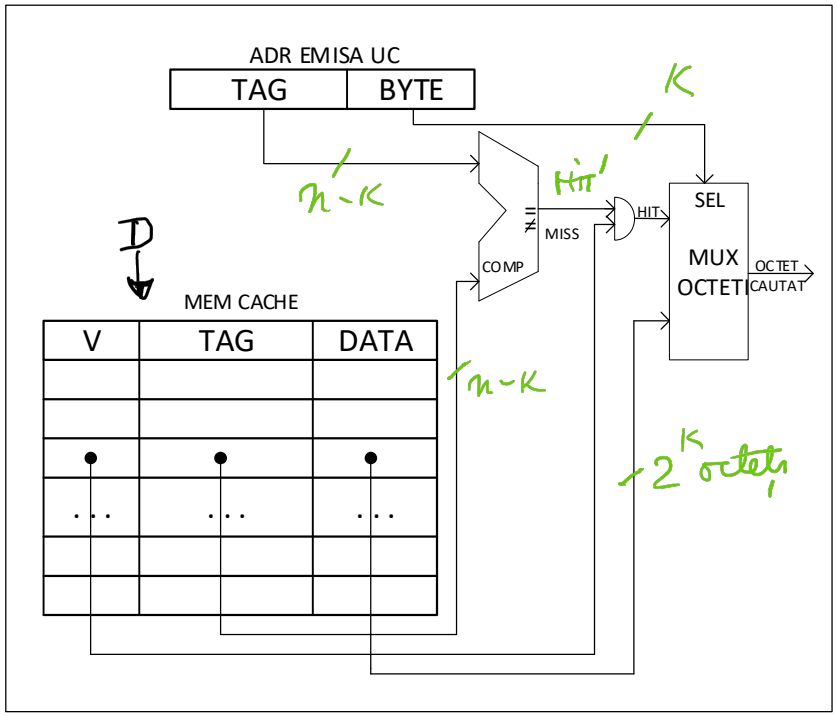
n - linii adrese în MM

2^n - locații în MM

2^K - octeți într-un slot CACHE (bloc de octeți în MM)

2^{n-K} - blocuri de 2^K octeți în MM

Cautarea in memoria cache organizata FULLY ASSOCIATIVE



Problema. Intr-un sistem de calcul memoria principala are organizarea 1M x 8. In acelasi sistem de calcul exista si memorie cache organizata FA (fully associative) cu un numar de 16k sloturi, in fiecare slot retinandu-se 8 octeti.

1) Care este formatul adresei emise de UC, asa cum este vazuta de cache?

2) Care este formatul campurilor din care este alcatuit un slot al cache-ului?

3) Care este organizarea si capacitatea cache-ului?

Solutie

1). Adresa emisa de UC are 20 biti (1M = 2^20). Cache-ul vede aceasta adresa ca fiind formata din 2 campuri (TAG si BYTE). Cum intr-un slot sunt 8 octeti (=2^3) rezulta k = 3, adica dimensiunea in biti a campului BYTE. Campul TAG va avea 20 – 3= 17 biti.

2). Slot-ul din cache contin campurile V (1 bit), D (1 bit), TAG (17 biti) si DATA (8 octeti = 8 x 8 = 64 biti); in total 83 de biti.

3). Organizarea cache = nr sloturi x continut in biti slot = 16k x 83.
Capacitatea cache = 16k x 83 = 16384 x 83 = 1359872 biti

2^4 locatii = 16 octeti → Block - 16 oct

24 biti - mag adrese

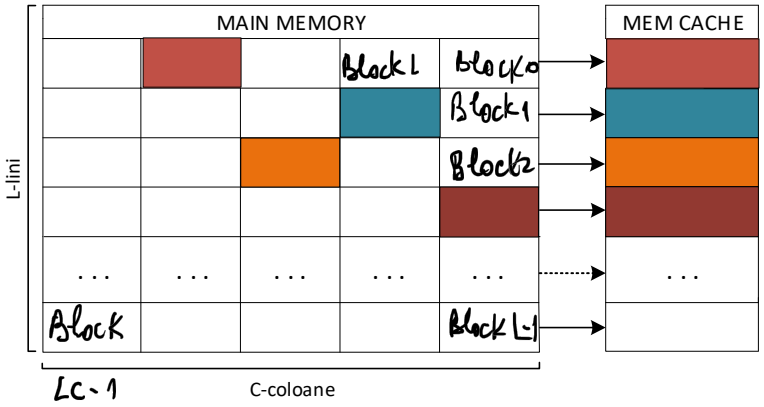
$A_{23} A_{22} A_{21} A_{20} \cdots\cdots A_7 A_6 A_5 A_4 \mid A_3 A_2 A_1 A_0$

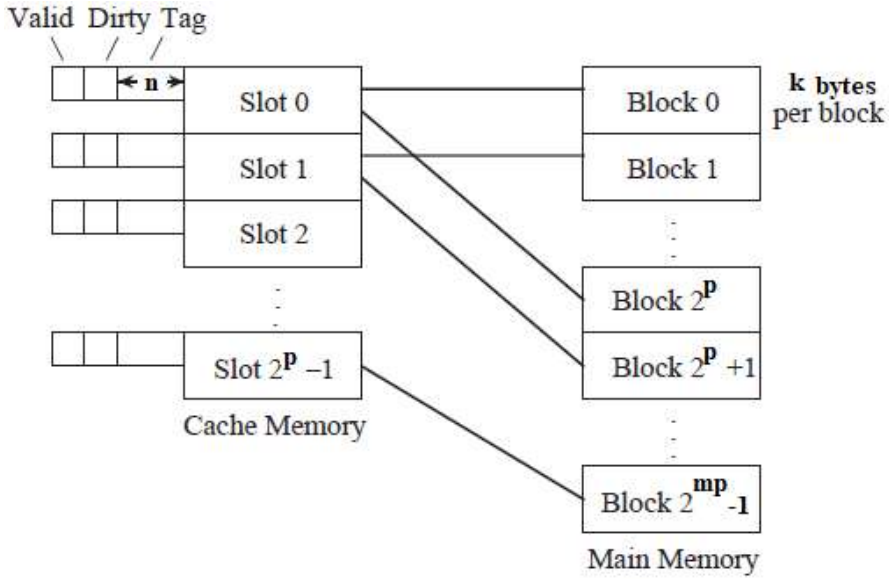
20 biti TAG | Byte

Adr. CPU (24 biti) → TAG (20 biti) + Byte (4 biti)

END 24 APR 2025

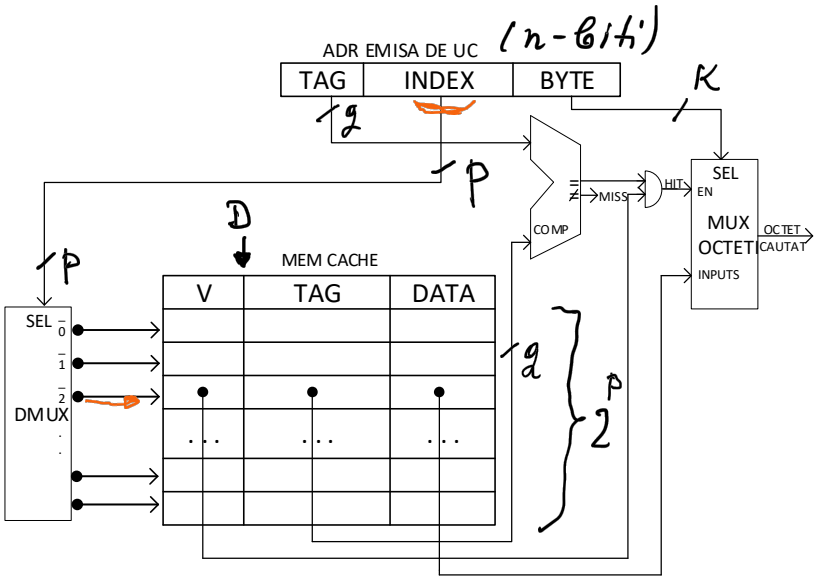
Memoria Cache organizata DIRECT MEMORY MAPPED (translatare directa din memorie)





Memoria principala este vazuta de cache ca fomand o matrice cu L linii si C coloane, numarul de linii fiind egal cu numarul de sloturi din cache.
Intr-o celula a acestei matrici se afla un bloc de octeti, numarul acestor octeti fiind similar cu numarul de octeti care formeaza un bloc de date pastrat intr-un slot al cache-ului.
Daca sunt L sloturi in cache, intr-un slot al cache-ului organizat cu translatare directa se pot afla blocuri din memoria principala care au o adresa cu acelasi **modulo L** (totusi doar unul singur).

Cautarea in memoria cache organizata DIRECT MEMORY MAPPED



- Cei „p” biti ai campului INDEX din adresa emisa de UC se aplica intrarilor de selectie de la DMUX-ul de tip 1: 2^p activandu-se o iesire corespunzatoare a acestuia; numarul de iesiri ale DMUX-ului coincide cu numarul sloturilor din cadrul cache-ului; activarea unei iesiri a DMUX-ului indica slotul in care se regaseste informatia cautata din memoria principala.
- Urmeaza sa se preia continutul campului TAG din slotul regasit al cache-ului si are loc o comparatie cu continutul campului TAG din adresa emisa de UC; din campul TAG are „q” biti si deci comparatorul permite comparatia a doua operatii pe „q” biti;
- Rezultatul comparatiei poate indica neidentitatea operanzilor (cazul MISS) sau identitatea acestora;
- In situatia identitatii are loc si verificarea bitului V din slotul regasit al cache-ului folosind o poarta SI, iar daca V = 1 la iesirea portii SI se obtine HIT (HIT = 1 activ).
- HIT = 1 activeaza un MUX de octeti care primeste pe intrarile sale continutul campului DATA din slotul regasit al cache-ului. Folosind campul BYTE din adresa emisa de UC ca informatie de selectie pentru MUX-ul de octeti, se obtine octetul cautat.

Adresa emisă de UC = n biți

$n = q + p + k$

CACHE-ul are 2^p - sloturi!

Aplicatie : Intr-un sistem de calcul, capacitatea de adresare a procesorului este 1 MB, in acelasi sistem de calcul o memorie cache, avand 32k sloturi, are campul de date dintr-un slot dimensionat pentru un bloc de 8 octeti. Se cere:

- Formatul adresei emise de UC, asa cum aceasta este vazuta de cache-ul organizat cu translatare directa din memorie (Direct Memory Mapped);
- Formatul matricii memoriei principale cu care procesorul opereaza, considerand ca aceasta ocupa tot spatiul de adresare al procesorului (asa cum aceasta este vazuta de procesor);
- Formatul unui slot din cache-ul organizat cu translatare directa din memorie;
- Organizarea si dimensiunea cache-ului.

Solutie: a) Este nevoie de precizarea numarului de biti a campurilor din adresa emisa de UC (campurile INDEX, BYTE respectiv TAG).

1MB = 1M x 8b (capacitatea de adresare a UC)

1M = 2^{20} - 20 de biti ai adresei emise de UC (n = 20).

Cum sunt 32k sloturi, adica 32k =$2^5 * 2^{10} = 2^{15}$ rezulta 15 reprezentand dimensiunea in biti a campului INDEX (p = 15). Cum intr-un slot sunt 8 octeti, adica 2^3 rezulta 3 – dimensiunea campului BYTE (k = 3). Urmeaza ca dimensiunea campului TAG este data de expresia 20-15-3=2 biti (q = n – p – k = 2).

b) Numarul de linii al matricii memoriei principale este egal cu numarul de sloturi = 32k = $2^5 * 2^{10} = 2^{15}$ linii. Notam numarul de coloane cu 2^c.

Intr-o celula a acestei matrici sunt 2^3 octeti. Cum organizarea memoriei principale este 2^{20} x 8 (1MB = 2^{20} octeti), urmeaza ca

$2^{20} = 2^{15} * 2^3 * 2^c => 2^{20} = 2^{18} * 2^c => 20 = 18 + c => c = 2$ rezulta ca sunt $2^2 = 4$ coloane cu 32k linii si cu o celula avand 8 octeti.

c) Un slot al cache-ului este format din campurile:

- V – 1 bit
- D – 1 bit
- TAG – 2 biti
- DATA – 8 octeti = 8 x 8 = 64 de biti

END 08 MAI 2025

d) Organizare cache: 32k x 68 Dimensiune cache: $2^5 * 2^{10} * 68 = 2^{15} * 68$ biti